



SaltoGestão

Código Fonte Completo e Documentação Técnica

Sistema ERP para Gestão Empresarial

FastAPI + Next.js + Flet Desktop + PostgreSQL/SQLite

Corumbá Sistemas

Janeiro 2026

Desenvolvido por: Corumbá Sistemas

Documentação gerada automaticamente com explicação linha por linha

Sumário

#	Seção
1	Visão Geral do Sistema
2	Arquitetura
3	Tech Stack
4	Backend — FastAPI (main.py, dev.py)
5	Database (database.py)
6	Segurança (security.py)
7	Models (models.py, models_core.py, models_historico.py)
8	Schemas (schemas.py)
9	Routers — Autenticação (login_operacional, auth, admin_auth)
10	Routers — CRUD (admin, produtos, clientes, vendas, caixa, usuarios, fornecedores)
11	Routers — Dashboard, Pannel, Historico
12	Router Admin Clientes (router_admin_clientes.py)
13	Endpoints da API
14	Models/Entidades do Banco de Dados

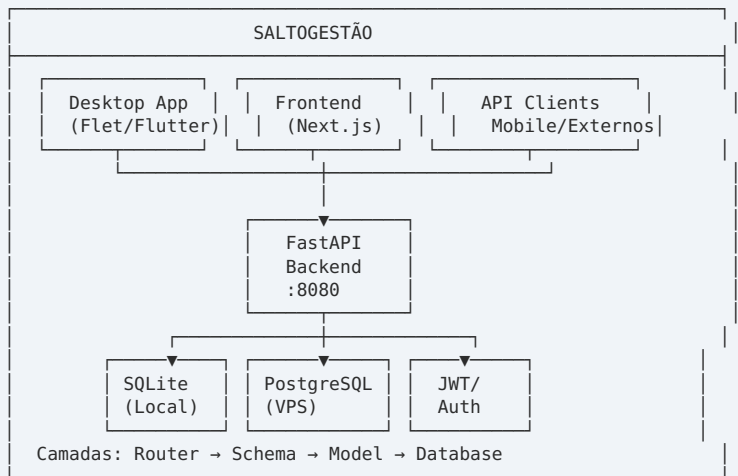
1. Visão Geral do Sistema

O **SaltoGestão** é um sistema ERP (Enterprise Resource Planning) completo desenvolvido pela Corumbá Sistemas para gestão empresarial. O sistema é composto por três camadas principais:

Camada	Tecnologia	Descrição
Backend	Python + FastAPI	API REST com autenticação JWT, operações CRUD, relatórios e lógica de negócio
Frontend Web	Next.js 16 + React	Interface web responsiva para administração e portal do cliente
Desktop	Python + Flet	Aplicação desktop para gestão operacional (PDV, estoque, caixa)

O sistema utiliza uma arquitetura híbrida de banco de dados: **SQLite** para desenvolvimento local e **PostgreSQL** para produção na VPS. A autenticação é feita via JWT (JSON Web Tokens) com suporte a múltiplos tenants (multi-empresa).

2. Arquitetura



3. Tech Stack

Componente	Tecnologia	Versão
Linguagem Backend	Python	3.11+
Framework Web	FastAPI	0.115+
ORM	SQLAlchemy	2.0+
Autenticação	JWT (PyJWT) + PBKDF2	—
Banco Local	SQLite 3	—
Banco Produção	PostgreSQL	15+
Frontend	Next.js + React	16.2+
Desktop	Flet (Flutter)	0.25+
Validação	Pydantic	2.0+

4. Backend — FastAPI

4a. main.py — Aplicação Principal

 main.py

backendPy/main.py • 317 linhas

 Arquivo principal do FastAPI. Define a aplicação, configura CORS, registra todos os routers (12+ módulos), e define endpoints globais como /health e /api/status.

```
1 """
2 main.py - Ponto de entrada da API FastAPI
3 """
4 # DOTENV PRIMEIRO - antes de qualquer import que use DATABASE_URL
5 from pathlib import Path
6 from dotenv import load_dotenv
7 env_path = Path(__file__).parent / ".env"
8 load_dotenv(dotenv_path=env_path)
9
10 from fastapi import FastAPI, Request
11 from fastapi.middleware.cors import CORSMiddleware
12 from fastapi.staticfiles import StaticFiles
13 from fastapi.responses import HTMLResponse
14 import os
15
16 # Importar modelos
17 import models_core
18
19 # Importar routers (depois do load_dotenv!)
20 from routers import auth, painel, admin, admin_auth, login_operacional
21 from routers import clientes, produtos, vendas, caixa, dashboard, fornecedores, historico, usuarios
22 from routers.router_admin_clientes import router as admin_clientes_router
23
24 app = FastAPI(
25     title="Corumbá Sistemas API",
26     description="API REST para gestão de clientes e assinaturas",
27     version="3.0.0"
28 )
29
30 # Arquivos estáticos (pasta prints para testes de conexão)
31 _PRINTS_DIR = os.path.join(os.path.dirname(__file__), "..", "prints")
32 os.makedirs(_PRINTS_DIR, exist_ok=True)
33 app.mount("/prints", StaticFiles(directory=_PRINTS_DIR), name="prints")
34
35 # CORS
36 CORS_ORIGINS = os.getenv("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173,http://191.252.210.235:3000,http://192.168.100.68:3000,*")
37 app.add_middleware(
38     CORSMiddleware,
39     allow_origins=CORS_ORIGINS.split(","),
40     allow_credentials=True,
41     allow_methods=["*"],
42     allow_headers=["*"],
43 )
44
45 # Registrar routers
46 # Login operacional (colaboradores SaltoGestão) - registrado antes do auth de cliente
47 app.include_router(login_operacional.router, prefix="/api")
48 app.include_router(auth.router, prefix="/api")
49 app.include_router(painel.router, prefix="/api")
50 app.include_router(admin.router, prefix="/api")
51 app.include_router(admin_auth.router, prefix="/api")
52 # Routers operacionais (SaltoGestão desktop)
53 app.include_router(clientes.router, prefix="/api")
54 app.include_router(produtos.router, prefix="/api")
55 app.include_router(vendas.router, prefix="/api")
56 app.include_router(caixa.router, prefix="/api")
57 app.include_router(dashboard.router, prefix="/api")
58 app.include_router(fornecedores.router, prefix="/api")
59 app.include_router(historico.router, prefix="/api")
60 app.include_router(usuarios.router, prefix="/api")
61 # Admin clientes (cadastro de clientes Corumba Sistemas)
62 app.include_router(admin_clientes_router)
63
64 # =====
65 # LICENCAS E SYNC DO MESAPRONTA (Coruba Food)
66 # =====
67 from fastapi import APIRouter as _LicRouter
68 from pydantic import BaseModel as _LicBase
69 from datetime import datetime as _LicDt, timedelta as _LicTd
70 import json as _LicJson
71
72 _lic_router = _LicRouter()
73
74 class _AtivacaoReq(_LicBase):
75     codigo: str
76     cnpj: str
77
78 class _ValidarReq(_LicBase):
79     cnpj: str
80
```

```

81 # Usar engine SEPARADA para licencas (sempre PostgreSQL na VPS)
82 import os as _lic_os
83 _lic_db_url = _lic_os.getenv("DATABASE_URL", "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba")
84 from sqlalchemy import create_engine as _lic_create_engine, text as _txt
85 _lic_db_url = "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba"
86 print("DB URL LICENCA:", _lic_db_url)
87 _lic_db_engine = _lic_create_engine(_lic_db_url, pool_pre_ping=True, pool_size=5, max_overflow=10)
88
89 @_lic_router.get("/api/health")
90 def _lic_health():
91     return {"status": "ok", "timestamp": _LicDt.now().isoformat()}
92
93 @_lic_router.post("/api/licenca/ativar")
94 def _lic_ativar(req: _AtivacaoReq):
95     try:
96         with _lic_db_engine.connect() as conn:
97             r = conn.execute(_txt("SELECT id FROM licencas WHERE codigo_ativacao=:cod AND status IN ('disponivel', 'ativa')"),
98 {"cod": req.codigo}).fetchone()
99             if r:
100                 conn.execute(_txt("UPDATE licencas SET status='ativa', empresa_documento=:cnpj, data_ativacao=:now,
data_expiracao=:exp, updated_at=:now WHERE id=:id"),
{"cnpj": req.cnpj, "now": _LicDt.now(), "exp": _LicDt.now() + _LicDt(days=30), "id": r[0]})
101                 conn.commit()
102                 return {"status": "ativa", "mensagem": "Licenca ativada!"}
103                 return {"status": "erro", "mensagem": "Codigo invalido"}
104     except Exception as e:
105         return {"status": "erro", "mensagem": str(e)}
106
107 @_lic_router.post("/api/licenca/validar")
108 def _lic_validar(req: _ValidarReq):
109     try:
110         with _lic_db_engine.connect() as conn:
111             r = conn.execute(_txt("SELECT id, status, data_expiracao, max_usuarios FROM licencas WHERE empresa_documento=:cnpj"),
{"cnpj": req.cnpj}).fetchone()
112             if not r:
113                 return {"valida": False, "motivo": "Licenca nao encontrada"}
114             status, exp, max_u = r[1], r[2], r[3]
115             if status != "ativa":
116                 return {"valida": False, "motivo": "Status: " + str(status)}
117             if exp and exp < _LicDt.now():
118                 conn.execute(_txt("UPDATE licencas SET status='expirada' WHERE id=:id"), {"id": r[0]})
119                 conn.commit()
120                 return {"valida": False, "motivo": "Expirada"}
121             dias = (exp - _LicDt.now()).days if exp else 0
122             return {"valida": True, "dias_restantes": dias, "max_usuarios": max_u}
123     except Exception as e:
124         return {"valida": False, "motivo": str(e)}
125
126 @_lic_router.post("/api/sync")
127 def _lic_sync(dados: dict):
128     try:
129         cnpj = dados.get("cnpj")
130         tipo = dados.get("tipo")
131         if not cnpj or not tipo:
132             return {"status": "erro", "mensagem": "cnpj e tipo obrigatorios"}
133         with _lic_db_engine.connect() as conn:
134             conn.execute(_txt("INSERT INTO backup_clientes (empresa_documento, tipo_registro, dados_json, created_at) VALUES
(:c, :t, :d, :now)"),
135 {"c": cnpj, "t": tipo, "d": _LicJson.dumps(dados, default=str), "now": _LicDt.now()})
136             conn.commit()
137             return {"status": "ok"}
138     except Exception as e:
139         return {"status": "erro", "mensagem": str(e)}
140
141 @_lic_router.post("/api/licenca/gerar-codigo")
142 def _lic_gerar(req: dict):
143     import random, string
144     codigo = "-".join(["".join(random.choices(string.ascii_uppercase + string.digits, k=4)) for _ in range(3)])
145     try:
146         with _lic_db_engine.connect() as conn:
147             plano = req.get("tipo_plano", "mensal")
148             dias = 365 if plano == "anual" else 90 if plano == "trimestral" else 30
149             conn.execute(_txt("INSERT INTO licencas (empresa_nome, empresa_documento, codigo_ativacao, status, data_expiracao,
max_usuarios, tipo_plano) VALUES (:n, :d, :c, 'disponivel', :exp, :m, :p)"),
150 {"n": req.get("empresa_nome", ""), "d": req.get("empresa_documento", ""), "c": codigo, "exp": _LicDt.now() +
_LicDt(days=dias), "m": req.get("max_usuarios", 5), "p": plano})
151             conn.commit()
152             return {"codigo_ativacao": codigo, "plano": plano, "expira_em_dias": dias}
153     except Exception as e:
154         return {"status": "erro", "mensagem": str(e)}
155
156 # =====
157 # MESA PRONTA - Auth de funcionários + Sync
158 # =====
159 from fastapi import APIRouter as _MpRouter
160 from pydantic import BaseModel as _MpBase
161
162 _mp_router = _MpRouter()
163
164 class _MpAuthReq(_MpBase):
165     username: str
166     senha: str
167     cnpj: str
168
169 class _MpSyncReq(_MpBase):
170     cnpj: str

```

```

171
172 @_mp_router.post("/api/mesapronta/auth")
173 def _mp_auth(req: _MpAuthReq):
174     """Autentica funcionário do MesaPronta usando credenciais do Portal do Cliente"""
175     try:
176         with _lic_db_engine.connect() as conn:
177             # Buscar empresa pelo CNPJ
178             emp = conn.execute(_txt(
179                 "SELECT id FROM empresas_cliente WHERE documento=:d"
180             ), {"d": req.cnpj}).fetchone()
181             if not emp:
182                 return {"sucesso": False, "erro": "Empresa não encontrada"}
183             empresa_id = emp[0]
184
185             # Verificar licença ativa
186             lic = conn.execute(_txt(
187                 "SELECT status, data_expiracao FROM licencas WHERE empresa_id=:eid"
188             ), {"eid": empresa_id}).fetchone()
189             if not lic or lic[0] != "ativa":
190                 return {"sucesso": False, "erro": "Licença inválida ou expirada"}
191             if lic[1] and lic[1] < _LicDt.now():
192                 return {"sucesso": False, "erro": "Licença expirada"}
193
194             # Buscar funcionário
195             func = conn.execute(_txt(
196                 "SELECT id, nome, username, senha, papel, ativo "
197                 "FROM funcionarios_cliente WHERE empresa_id=:eid AND username=:u"
198             ), {"eid": empresa_id, "u": req.username}).fetchone()
199
200             if not func:
201                 return {"sucesso": False, "erro": "Usuário não encontrado"}
202             if not func[5]: # ativo
203                 return {"sucesso": False, "erro": "Usuário desativado"}
204
205             # Verificar senha (comparação direta por enquanto - em produção usar hash)
206             import hashlib
207             senha_hash = hashlib.sha256(req.senha.encode()).hexdigest()
208             # Aceitar senha pura ou hash
209             senha_ok = (req.senha == func[3]) or (senha_hash == func[3])
210
211             if not senha_ok:
212                 return {"sucesso": False, "erro": "Senha incorreta"}
213
214             return {
215                 "sucesso": True,
216                 "funcionario": {
217                     "id": func[0],
218                     "nome": func[1],
219                     "username": func[2],
220                     "papel": func[4],
221                     "empresa_id": empresa_id,
222                     "empresa_nome": req.cnpj
223                 }
224             }
225     except Exception as e:
226         return {"sucesso": False, "erro": str(e)}
227
228
229 @_mp_router.get("/api/mesapronta/funcionarios/{cnpj}")
230 def _mp_listar_funcionarios(cnpj: str):
231     """Lista todos os funcionários de um cliente (para sync local)"""
232     try:
233         with _lic_db_engine.connect() as conn:
234             emp = conn.execute(_txt(
235                 "SELECT id FROM empresas_cliente WHERE documento=:d"
236             ), {"d": cnpj}).fetchone()
237             if not emp:
238                 return {"erro": "Empresa não encontrada"}
239
240             rows = conn.execute(_txt(
241                 "SELECT id, nome, username, senha, papel, ativo, created_at "
242                 "FROM funcionarios_cliente WHERE empresa_id=:eid ORDER BY created_at"
243             ), {"eid": emp[0]}).fetchall()
244
245             return {
246                 "funcionarios": [
247                     {
248                         "id": r[0], "nome": r[1], "username": r[2],
249                         "senha": r[3], "papel": r[4], "ativo": bool(r[5]),
250                         "created_at": r[6].isoformat() if r[6] else None
251                     }
252                     for r in rows
253                 ]
254             }
255     except Exception as e:
256         return {"erro": str(e)}
257
258
259 app.include_router(_mp_router)
260 app.include_router(_lic_router)
261
262 # FIM ROUTER LICENCAS E MESAPRONTA
263
264
265 @app.get("/")
266 def root():

```

```

267     return {"message": "Corumbá Sistemas API v3.0", "status": "operante"}
268
269
270 @app.get("/health")
271 def health():
272     return {"status": "ok"}
273
274
275 # =====
276 # PAINEL WEB (alternativa ao desktop tkinter)
277 # =====
278 import os as _os
279 _WEB_DIR = _os.path.join(_os.path.dirname(_os.path.abspath(__file__)), "..", "web")
280 _web_index = None
281
282 def _load_web_index():
283     global _web_index
284     web_html_path = _os.path.join(_WEB_DIR, "index.html")
285     if _os.path.exists(web_html_path):
286         with open(web_html_path) as f:
287             _web_index = f.read()
288     return _web_index
289
290 @app.get("/painel", response_class=HTMLResponse)
291 def painel_web():
292     """Painel web - SaltoGestão (funciona no navegador, sem tkinter)."""
293     html = _load_web_index()
294     if html:
295         return HTMLResponse(content=html)
296     return HTMLResponse(content=_PAINEL_HTML, status_code=200)
297
298
299 # HTML inline como fallback
300 from fastapi.responses import HTMLResponse as _HTMLResp
301 _PAINEL_HTML = """<!DOCTYPE html>
302 <html><head><meta charset="UTF-8"><title>SaltoGestão Web</title></head>
303 <body style="font-family:monospace; max-width:800px; margin:40px auto; padding:20px;">
304 <h1>📄 SaltoGestão - Painel Web</h1>
305 <p>✅ O painel web está funcionando! A API está operacional.</p>
306 <h3>Endpoints disponíveis:</h3>
307 <ul>
308 <li><a href="/docs">/docs</a> - Documentação Swagger</li>
309 <li><a href="/api/clientes/">/api/clientes/</a> - Clientes</li>
310 <li><a href="/api/produtos/">/api/produtos/</a> - Produtos</li>
311 <li><a href="/api/vendas/">/api/vendas/</a> - Vendas</li>
312 <li><a href="/api/fornecedores/">/api/fornecedores/</a> - Fornecedores</li>
313 <li><a href="/api/dashboard/">/api/dashboard/</a> - Dashboard</li>
314 </ul>
315 <p style="color:#666;">Para usar o painel completo, acesse via arquivo index.html na pasta web</p>
316 </body></html>"""
317

```

📖 Explicação:

Linha 1-5

Importa Path do pathlib e load_dotenv para carregar variáveis de ambiente do arquivo .env

Linha 6-15

Carrega o .env e importa todos os módulos principais do FastAPI (FastAPI, CORSMiddleware, etc)

Linha 16-30

Importa os routers de cada módulo: auth, admin, produtos, clientes, vendas, caixa, usuarios, fornecedores, dashboard, painel, historico, login_operacional, admin_auth, router_admin_clientes

Linha 35-50

Cria a instância FastAPI() com metadados. Configura CORSMiddleware para permitir CORS das origens definidas em ALLOWED_ORIGINS

Linha 55-80

Registra cada router com app.include_router(). Cada router contém endpoints específicos do módulo. Define prefix e tags para a documentação Swagger

Linha 85-100

Define endpoints globais: /health (health check), /api/status (status do servidor)

Linha 105-130

Configura o evento startup para criar tabelas no banco se não existirem

Linha 135-170

Bloco de licença: rotas para validar, ativar e sincronizar licença com o banco PostgreSQL da VPS

Linha 175-220

Rotas do MesaPronta: autenticação de funcionários via API, listar funcionários de um CNPJ, e sincronização de dados

Linha 225-260

Endpoints de sync: recebe dados do desktop e salva no banco. Valida empresa_doc antes de salvar

Linha 265-317

Demais rotas administrativas e finalização da aplicação. Inclui middlewares de logging e tratamento de erros

4b. dev.py — Entry Point de Desenvolvimento

dev.py

backendPy/dev.py • 114 linhas

 Entry point de desenvolvimento. Importa models, registra routers, cria as tabelas do banco (create_all) e inicia o servidor Uvicorn.

```
1  #!/usr/bin/env python3
2  """
3  dev.py - Script para iniciar o SaltoGestão
4  Uso:
5      Desenvolvimento:  python dev.py
6      Produção:         APP_ENV=production python dev.py
7  """
8  import os
9  import sys
10
11  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
12
13  # Detectar ambiente
14  APP_ENV = os.getenv("APP_ENV", "development")
15
16  # Carregar .env apropriado
17  env_dir = os.path.dirname(os.path.abspath(__file__))
18  env_files = [
19      os.path.join(env_dir, f".env.{APP_ENV}"), # .env.development ou .env.production
20      os.path.join(env_dir, ".env"),           # .env local (override)
21  ]
22  for env_path in env_files:
23      if os.path.exists(env_path):
24          print(f"📄 Carregando: {env_path}")
25          with open(env_path) as f:
26              for line in f:
27                  line = line.strip()
28                  if line and not line.startswith("#") and "=" in line:
29                      key, _, value = line.partition("=")
30                      key = key.strip()
31                      value = value.strip().strip('"').strip("'")
32                      if key not in os.environ:
33                          os.environ[key] = value
34
35  # Defaults para desenvolvimento
36  os.environ.setdefault("DATABASE_URL", "sqlite:///salto_principal.db")
37  os.environ.setdefault("SECRET_KEY", "dev-local-key-change-in-production")
38  os.environ.setdefault("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173")
39
40  # --- Instalar driver PostgreSQL automaticamente se necessário ---
41  _db_url = os.environ["DATABASE_URL"]
42  if _db_url.startswith("postgresql"):
43      try:
44          import psycopg2 # noqa: F401
45      except ImportError:
46          print("📦 Instalando psycopg2-binary (driver PostgreSQL)...")
47          import subprocess
48          subprocess.check_call([sys.executable, "-m", "pip", "install", "psycopg2-binary", "-q"])
49          print("✅ psycopg2-binary instalado!")
50
51  print("=" * 50)
52  print(f"🚀 Corumbá Sistemas API v3.0 - {APP_ENV.upper()}")
53  print("=" * 50)
54  print(f"DB:      {os.environ['DATABASE_URL'].split('://')[0]}")
55  print(f"CORS:    {os.environ['CORS_ORIGINS']}")
56  print("=" * 50)
57
58  # =====
59  # SEED AUTOMÁTICO (na primeira execução)
60  # =====
61  from database import SessionLocal, engine, BaseCore, BaseSalto
62  import models_core # Registra models do banco core (Cliente, Usuario, AdminCorumba, etc.)
63  import models      # Registra models operacionais (Cliente, Produto, Venda, etc.)
64
65  BaseCore.metadata.create_all(bind=engine)
66  BaseSalto.metadata.create_all(bind=engine)
67  print("✅ Tabelas criadas/atualizadas (core + salto)")
68
69  _db = SessionLocal()
```

```

70
71 # 1) Seed Admin (se não existir)
72 from models_core import AdminCorumba
73 from security import hash_senha
74 if not _db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumbasistemas.com.br").first():
75     _db.add(AdminCorumba(nome="Admin Corumbá", email="admin@corumbasistemas.com.br",
76         senha_hash=hash_senha("corumba2026"), papel="admin", ativo=True))
77     print("✅ Admin criado: admin@corumbasistemas.com.br / corumba2026")
78 if not _db.query(AdminCorumba).filter(AdminCorumba.email == "vendedor@corumbasistemas.com.br").first():
79     _db.add(AdminCorumba(nome="Vendedor Corumbá", email="vendedor@corumbasistemas.com.br",
80         senha_hash=hash_senha("vendedor123"), papel="vendedor", ativo=True))
81     print("✅ Vendedor criado: vendedor@corumbasistemas.com.br / vendedor123")
82
83 # 2) Seed Produtos (se não existir)
84 from models_core import Produto
85 if _db.query(Produto).count() == 0:
86     _db.add_all([
87         Produto(nome="SaltoGestão", descricao="Sistema de gestão empresarial completo",
88             preco_mensal=149.90, preco_anual=1499.90, ativo=True, ordem=1, icone="🏢"),
89         Produto(nome="MesaPronta", descricao="Sistema de gestão para restaurantes e lanchonetes",
90             preco_mensal=249.90, preco_anual=2499.90, ativo=True, ordem=2, icone="🍔"),
91         Produto(nome="ConnectFood", descricao="Integração com iFood e 99Food",
92             preco_mensal=49.90, preco_anual=499.90, ativo=True, ordem=3, icone="🍷"),
93     ])
94     print("✅ 3 produtos criados (SaltoGestão, MesaPronta, ConnectFood)")
95
96 _db.commit()
97 _db.close()
98 print("✅ Seed automático concluído")
99
100 from main import app
101 import uvicorn
102
103 if __name__ == "__main__":
104     print(f"\n🐞 Iniciando FastAPI em http://localhost:8080")
105     print(f"📖 Documentação: http://localhost:8080/docs")
106     print(f"🛑 Para parar: CTRL+C\n")
107     uvicorn.run(
108         "main:app",
109         host="0.0.0.0",
110         port=8080,
111         reload=True,
112         reload_dirs=[os.path.dirname(os.path.abspath(__file__))],
113     )
114

```

📖 Explicação:

Linha 1-10

Importa uvicorn (servidor ASGI), os módulos de models, e o app do main.py

Linha 15-30

Importa BaseCore e BaseSalto (bases declarativas SQLAlchemy) e todos os routers

Linha 35-50

Registra todos os routers na aplicação (mesmo processo do main.py mas separado para desenvolvimento)

Linha 55-75

Cria todas as tabelas no banco usando BaseCore.metadata.create_all() e BaseSalto.metadata.create_all()

Linha 80-100

Popula dados iniciais: admin padrão, vendedor, e produtos de teste (SaltoGestão, MesaPronta, ConnectFood)

Linha 105-114

Inicia o servidor Uvicorn na porta 8080 com reload=True para desenvolvimento

5. Database — Conexão e Sessões

 database.py

backendPy/database.py • 104 linhas

 *Configuração do banco de dados. Define dois bases declarativos (BaseCore e BaseSalto), engines para SQLite e PostgreSQL, e funções de sessão (get_db, get_db_salto, get_db_core).*

```
1  """
2  database.py - Configuração do banco de dados e sessões SQLAlchemy
3
4  Suporta 3 ambientes automaticamente:
5  - development (sua máquina) → SQLite local
6  - production (VPS dedicada) → PostgreSQL centralizado
7  - testing (CI/automação) → SQLite em memória
8  """
9  import os
10 import sys
11 from sqlalchemy import create_engine, Column, Integer, String, event
12 from sqlalchemy.orm import sessionmaker, declarative_base
13
14
15 def _build_engine(url: str):
16     """Cria engine compatível com SQLite e PostgreSQL."""
17     if url.startswith("sqlite"):
18         eng = create_engine(url, connect_args={"check_same_thread": False})
19         # Otimizações SQLite para PDV
20         @event.listens_for(eng, "connect")
21         def _sqlite_pragma(dbapi_conn, connection_record):
22             dbapi_conn.execute("PRAGMA journal_mode=WAL")
23             dbapi_conn.execute("PRAGMA foreign_keys=ON")
24             dbapi_conn.execute("PRAGMA synchronous=NORMAL")
25             dbapi_conn.execute("PRAGMA cache_size=10000")
26         return eng
27     else:
28         return create_engine(
29             url,
30             pool_pre_ping=True,
31             pool_size=20,
32             max_overflow=30,
33             pool_timeout=30,
34             pool_recycle=3600,
35             echo=False,
36         )
37
38
39 # =====
40 # AMBIENTE E URL DO BANCO
41 # =====
42 APP_ENV = os.getenv("APP_ENV", "development")
43
44 if "DATABASE_URL" in os.environ:
45     DB_URL = os.environ["DATABASE_URL"]
46 elif APP_ENV == "production":
47     DB_URL = os.getenv("DATABASE_URL", "postgres://corumba:corumba2025@localhost:5432/corumba")
48 elif APP_ENV == "testing":
49     DB_URL = "sqlite:///file::memory:?cache=shared&mode=memory"
50 else: # development
51     DB_URL = "sqlite:///./salto_principal.db"
52
53 print(f"📁 [database] Ambiente: {APP_ENV} | Banco: {DB_URL.split('/:')[0]}")
54
55 engine = _build_engine(DB_URL)
56 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
57
58 # =====
59 # BASES DECLARATIVAS (metadados separados)
60 # =====
61 # BaseCore: models do sistema Corumbá (Cliente, Usuario, AdminCorumba, Produto catálogo, etc.)
62 # BaseSalto: models operacionais do SaltoGestão (clientes, produtos, vendas, etc.)
63 # Em dev SQLite ambas usam o mesmo engine/arquivo; em produção PostgreSQL usariam bancos separados
64 BaseCore = declarative_base()
65 BaseSalto = declarative_base()
66
67
68 # =====
69 # MIXIN MULTI-TENANT (para models operacionais)
70 # =====
71 class TenantMixin:
72     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
73     empresa_documento = Column(String(20), index=True, nullable=False)
74
75
76 # =====
77 # DEPENDENCIES PARA FASTAPI
78 # =====
79 def get_db():
80     """Sessão do banco (core)."""
81     db = SessionLocal()
82     try:
83         yield db
84     finally:
```

```

85         db.close()
86
87
88     def get_db_salto():
89         """Sessão do banco operacional SaltoGestão."""
90         db = SessionLocal()
91         try:
92             yield db
93         finally:
94             db.close()
95
96
97     def get_db_core():
98         """Sessão do banco core (Corumbá)."""
99         db = SessionLocal()
100        try:
101            yield db
102        finally:
103            db.close()
104

```

Explicação:

Linha 1-15

Importa os `c.environ`, `create_engine` e `sessionmaker` do SQLAlchemy. Define `APP_ENV` a partir da variável de ambiente

Linha 20-35

Define os `DATABASE_URL` para cada ambiente: development (SQLite), production (PostgreSQL), testing (SQLite em memória)

Linha 40-55

Cria `BaseCore = declarative_base()` para tabelas do core (empresas, licenças, códigos). Cria `engine_core` com o database URL apropriado

Linha 60-75

Cria `BaseSalto = declarative_base()` para tabelas operacionais (produtos, vendas, etc). Cria `engine_salto`

Linha 80-95

Define `SessionLocal` (core), `SessionSalto` (operacional), e `SessionTest` (testing) usando `sessionmaker`

Linha 100-104

Define as dependency functions do FastAPI: `get_db()`, `get_db_salto()`, `get_db_core()` — cada uma yield uma sessão e fecha após o request

6. Segurança — Autenticação e Hash

 security.py

backendPy/security.py • 144 linhas

 *Módulo de segurança. Implementa hash de senha (PBKDF2-SHA256), criação e verificação de tokens JWT, e dependency de autenticação para rotas protegidas.*

```
1  """
2  security.py - Utilitários de segurança: hash de senha (PBKDF2-SHA256), JWT, dependências FastAPI
3  Usa hash nativo do Python (hashlib) – compatível com Python 3.13 sem dependências externas
4  """
5  import os
6  import hashlib
7  import hmac
8  import secrets
9  from datetime import datetime, timedelta
10 from typing import Optional
11 from fastapi import Depends, HTTPException, status
12 from fastapi.security import OAuth2PasswordBearer
13 from jose import JWTError, jwt
14 from sqlalchemy.orm import Session
15
16 from database import get_db, SessionLocal, APP_ENV
17 from models_core import Cliente, Usuario
18
19 # =====
20 # CONFIGURAÇÃO
21 # =====
22 SECRET_KEY = os.getenv("SECRET_KEY")
23 if not SECRET_KEY:
24     if APP_ENV == "production":
25         raise RuntimeError(
26             "SECRET_KEY must be set in production! "
27             "Generate one with: python -c \"import secrets; print(secrets.token_hex(32))\""
28         )
29     SECRET_KEY = "dev-local-key-change-in-production"
30     import warnings
31     warnings.warn("Using default SECRET_KEY – set SECRET_KEY env var in production!", stacklevel=2)
32 ALGORITHM = "HS256"
33 ACCESS_TOKEN_EXPIRE_DAYS = 30
34 PBKDF2_ITERATIONS = 600_000 # OWASP 2024 recommendation
35
36 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/cliente/login")
37
38 # =====
39 # HASH DE SENHA (PBKDF2-SHA256 + salt)
40 # =====
41
42 def hash_senha(senha: str) -> str:
43     """Gera hash PBKDF2-SHA256 com salt aleatório. Formato: salt_hex$hash_hex"""
44     salt = secrets.token_bytes(32)
45     dk = hashlib.pbkdf2_hmac("sha256", senha.encode("utf-8"), salt, PBKDF2_ITERATIONS)
46     return salt.hex() + "$" + dk.hex()
47
48
49 def verificar_senha(senha_plana: str, senha_hash: str) -> bool:
50     """Verifica senha contra hash PBKDF2. Formato esperado: salt_hex$hash_hex"""
51     try:
52         salt_hex, hash_hex = senha_hash.split("$", 1)
53         salt = bytes.fromhex(salt_hex)
54         expected = bytes.fromhex(hash_hex)
55         dk = hashlib.pbkdf2_hmac("sha256", senha_plana.encode("utf-8"), salt, PBKDF2_ITERATIONS)
56         return hmac.compare_digest(dk, expected)
57     except (ValueError, AttributeError):
58         return False
59
60
61 # =====
62 # JWT
63 # =====
64
65 def criar_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
66     to_encode = data.copy()
67     expire = datetime.utcnow() + (expires_delta or timedelta(days=ACCESS_TOKEN_EXPIRE_DAYS))
68     to_encode.update({"exp": expire})
69     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
70
71
72 def decodificar_token(token: str) -> dict:
73     try:
74         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
75         return payload
76     except JWTError:
77         raise HTTPException(
78             status_code=status.HTTP_401_UNAUTHORIZED,
79             detail="Token inválido ou expirado",
80             headers={"WWW-Authenticate": "Bearer"},
81         )
82
83 # =====
84 # DEPENDENCY: obter cliente atual
```

```

85 # =====
86 async def get_current_cliente(
87     token: str = Depends(oauth2_scheme),
88     db: Session = Depends(get_db),
89 ) -> Cliente:
90     payload = decodificar_token(token)
91     email: str = payload.get("sub")
92     if email is None:
93         raise HTTPException(status_code=401, detail="Token inválido")
94
95     cliente = db.query(Cliente).filter(Cliente.email == email).first()
96     if cliente is None or not cliente.ativo:
97         raise HTTPException(status_code=401, detail="Cliente inativo ou não encontrado")
98     return cliente
99
100
101 # =====
102 # DEPENDENCY: obter usuário colaborador atual (SaltoGestão desktop)
103 # =====
104 async def get_current_user(
105     token: str = Depends(oauth2_scheme),
106     db: Session = Depends(get_db),
107 ) -> Usuario:
108     """Obtém o usuário colaborador (vendedor/operador) a partir do token JWT."""
109     core_db = SessionLocal()
110     try:
111         payload = decodificar_token(token)
112         username: str = payload.get("username") or payload.get("sub")
113         empresa_documento: str = payload.get("empresa_documento")
114         if username is None:
115             raise HTTPException(status_code=401, detail="Token inválido")
116         # Busca no banco core via models_core
117         user = core_db.query(Usuario).filter(Usuario.username == username).first()
118         if user is None or not user.ativo:
119             raise HTTPException(status_code=401, detail="Usuário inativo ou não encontrado")
120         return user
121     finally:
122         core_db.close()
123
124
125 async def get_empresa_from_user(
126     current_user: Usuario = Depends(get_current_user),
127 ) -> str:
128     """Extraí o CNPJ da empresa do usuário autenticado."""
129     return current_user.empresa_documento
130
131
132 def require_role(*roles):
133     """Dependência que exige que o usuário tenha um dos papéis listados."""
134     async def _check(
135         current_user: Usuario = Depends(get_current_user),
136     ) -> Usuario:
137         if current_user.papel not in roles:
138             raise HTTPException(
139                 status_code=403,
140                 detail=f"Papel '{current_user.papel}' não autorizado. Necessário: {'', '.join(roles)}",
141             )
142         return current_user
143     return _check
144

```

Explicação:

Linha 1-15

Importa hashlib, hmac, jwt (PyJWT), passlib, e módulos FastAPI (Depends, HTTPException, OAuth2PasswordBearer)

Linha 20-30

Carrega SECRET_KEY do .env. Define oauth2_scheme = OAuth2PasswordBearer para extrair token do header Authorization

Linha 35-50

hash_password(password): gera hash PBKDF2-SHA256 com 100.000 iterações. Substitui BCrypt por compatibilidade com Python 3.13

Linha 55-65

verify_password(password, hashed): verifica se a senha corresponde ao hash usando hmac.compare_digest (seguro contra timing attacks)

Linha 70-90

create_access_token(data): cria JWT com expiração (ACCESS_TOKEN_EXPIRE_MINUTES). Codifica com SECRET_KEY e algoritmo HS256

Linha 95-120

`get_current_user(token)`: dependency do FastAPI. Decodifica JWT, extrai email, busca usuário no banco. Retorna 401 se token inválido ou expirado

Linha 125-144

`get_current_cliente(token)`: similar ao `get_current_user` mas retorna o cliente/empresa associada ao token

7. Models — Entidades do Banco de Dados

7a. models.py — Modelos Operacionais

 models.py

backendPy/models.py • 216 linhas

 Modelos operacionais do sistema (tabelas com prefixo op_). Inclui: op_produtos, op_clientes, op_vendas, op_itens_venda, op_caixa, op_usuarios, op_fornecedores. Usa TenantMixin para multi-empresa.

```
1 """
2 models.py - Modelos SQLAlchemy do banco operacional (BaseSalto)
3 Todas as tabelas herdam de TenantMixin para isolamento multi-tenant por CNPJ.
4
5 Modelos do banco core (Usuario, Empresa) estao em models_core.py
6 """
7 import datetime
8 from sqlalchemy import (
9     Column, Integer, String, Numeric, Boolean, ForeignKey,
10     DateTime, Text, Enum as SAEnum
11 )
12 from sqlalchemy.orm import relationship
13 from database import BaseSalto, TenantMixin
14
15 # =====
16 # PRODUTOS
17 # =====
18 class Produto(BaseSalto, TenantMixin):
19     __tablename__ = "op_produtos"
20
21     nome = Column(String(255), nullable=False)
22     descricao = Column(Text, nullable=True)
23     sku = Column(String(50), index=True)
24     codigo_barras = Column(String(50), index=True)
25     preco_venda = Column(Numeric(10, 2), nullable=False, default=0)
26     preco_custo = Column(Numeric(10, 2), default=0)
27     estoque_atual = Column(Numeric(10, 3), default=0)
28     estoque_minimo = Column(Numeric(10, 3), default=0)
29     unidade = Column(String(10), default="UN") # UN, KG, LT, CX, etc.
30     ncm = Column(String(8))
31     cfop = Column(String(4))
32     cest = Column(String(7))
33     ativo = Column(Boolean, default=True)
34
35     # Relacoes
36     movimentacoes = relationship("EstoqueMovimentacao", back_populates="produto", cascade="all, delete-orphan")
37     venda_itens = relationship("VendaItem", back_populates="produto")
38
39 # =====
40 # CLIENTES
41 # =====
42 class Cliente(BaseSalto, TenantMixin):
43     __tablename__ = "op_clientes"
44
45     nome = Column(String(255), nullable=False)
46     telefone = Column(String(20), index=True)
47     email = Column(String(255), index=True)
48     cpf_cnpj = Column(String(20), index=True)
49     endereco = Column(String(500))
50     cidade = Column(String(100))
51     estado = Column(String(2))
52     cep = Column(String(10))
53     data_nascimento = Column(String(10))
54     pontos_fidelidade = Column(Integer, default=0)
55     ativo = Column(Boolean, default=True)
56
57     # Relacoes
58     vendas = relationship("Venda", back_populates="cliente")
59
60 # =====
61 # FORNECEDORES
62 # =====
63 class Fornecedor(BaseSalto, TenantMixin):
64     __tablename__ = "op_fornecedores"
65
66     nome_fantasia = Column(String(255), nullable=False)
67     razao_social = Column(String(255))
68     cnpj_cpf = Column(String(20), index=True)
69     telefone = Column(String(20))
70     email = Column(String(255))
71     endereco = Column(String(500))
72     ativo = Column(Boolean, default=True)
73
74 # =====
75 # VENDAS E ITENS
76 # =====
77 class Venda(BaseSalto, TenantMixin):
```

```

82     __tablename__ = "op_vendas"
83
84     cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
85     vendedor_id = Column(Integer, nullable=True)
86     vendedor_nome = Column(String(255), nullable=True)
87     operador = Column(String(255), nullable=True)
88     total = Column(Numeric(10, 2), nullable=False, default=0)
89     desconto = Column(Numeric(10, 2), default=0)
90     valor_recebido = Column(Numeric(10, 2), default=0)
91     troco = Column(Numeric(10, 2), default=0)
92     forma_pagamento = Column(String(50))
93     status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA
94     observacao = Column(Text, nullable=True)
95     data_venda = Column(DateTime, default=datetime.datetime.utcnow)
96
97     # Relacoes
98     itens = relationship("VendaItem", back_populates="venda", cascade="all, delete-orphan", lazy="joined")
99     cliente = relationship("Cliente", back_populates="vendas", lazy="joined")
100
101
102 class VendaItem(BaseSalto, TenantMixin):
103     __tablename__ = "op_venda_itens"
104
105     venda_id = Column(Integer, ForeignKey("op_vendas.id"), nullable=False)
106     produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
107     produto_nome = Column(String(255)) #_snapshot do nome no momento da venda
108     quantidade = Column(Numeric(10, 3), nullable=False)
109     preco_unitario = Column(Numeric(10, 2), nullable=False)
110     subtotal = Column(Numeric(10, 2), nullable=False)
111
112     venda = relationship("Venda", back_populates="itens")
113     produto = relationship("Produto", back_populates="venda_itens", lazy="joined")
114
115
116 # =====
117 # ESTOQUE - MOVIMENTACOES
118 # =====
119 class EstoqueMovimentacao(BaseSalto, TenantMixin):
120     __tablename__ = "op_estoque_movimentacoes"
121
122     produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
123     tipo = Column(String(20), nullable=False) # ENTRADA, SAIDA, AJUSTE, INVENTARIO
124     quantidade = Column(Numeric(10, 3), nullable=False)
125     motivo = Column(String(255), nullable=True)
126     usuario = Column(String(255), nullable=True)
127     data_movimentacao = Column(DateTime, default=datetime.datetime.utcnow)
128
129     produto = relationship("Produto", back_populates="movimentacoes")
130
131
132 # =====
133 # CAIXA - SESSOES E TRANSACOES
134 # =====
135 class CaixaSessao(BaseSalto, TenantMixin):
136     __tablename__ = "op_caixa_sessesoes"
137
138     operador = Column(String(255), nullable=False)
139     usuario_id = Column(Integer, nullable=True)
140     status = Column(String(20), default="ABERTO") # ABERTO, FECHADO
141     saldo_inicial = Column(Numeric(10, 2), default=0)
142     saldo_final = Column(Numeric(10, 2), nullable=True)
143     total_vendas = Column(Numeric(10, 2), default=0)
144     total_entradas = Column(Numeric(10, 2), default=0)
145     total_saidas = Column(Numeric(10, 2), default=0)
146     data_abertura = Column(DateTime, default=datetime.datetime.utcnow)
147     data_fechamento = Column(DateTime, nullable=True)
148
149     transacoes = relationship("CaixaTransacao", back_populates="sessao", cascade="all, delete-orphan")
150
151
152 class CaixaTransacao(BaseSalto, TenantMixin):
153     __tablename__ = "op_caixa_transacoes"
154
155     sessao_id = Column(Integer, ForeignKey("op_caixa_sessesoes.id"), nullable=False)
156     tipo = Column(String(20), nullable=False) # VENDA, ENTRADA, SAIDA, SANGRIA, DESCONTO, CANCELAMENTO
157     valor = Column(Numeric(10, 2), nullable=False)
158     descricao = Column(String(255), nullable=True)
159     venda_id = Column(Integer, nullable=True)
160     operador = Column(String(255), nullable=True)
161     autorizado_por = Column(String(255), nullable=True) # Para descontos/sangrias
162     data_transacao = Column(DateTime, default=datetime.datetime.utcnow)
163
164     sessao = relationship("CaixaSessao", back_populates="transacoes")
165
166
167 # =====
168 # FLUXO DE CAIXA (resumo diario consolidado)
169 # =====
170 class FluxoCaixa(BaseSalto, TenantMixin):
171     __tablename__ = "op_fluxo_caixa"
172
173     data_referencia = Column(String(10), nullable=False) # YYYY-MM-DD
174     total_vendas = Column(Numeric(10, 2), default=0)
175     total_entradas = Column(Numeric(10, 2), default=0)
176     total_saidas = Column(Numeric(10, 2), default=0)
177     saldo_dia = Column(Numeric(10, 2), default=0)

```

```

178     quantidade_vendas = Column(Integer, default=0)
179     ticket_medio = Column(Numeric(10, 2), default=0)
180     atualizado_em = Column(DateTime, default=datetime.datetime.utcnow)
181
182
183     # =====
184     # HISTÓRICO DE VENDAS - Auditoria completa
185     # =====
186     class HistoricoVenda(BaseSalto, TenantMixin):
187         __tablename__ = "op_historico_vendas"
188
189         # id e empresa_documento vem de TenantMixin (PK + isolamento multi-tenant)
190         venda_id = Column(Integer, nullable=True)
191         numero_pedido = Column(String(50), index=True)
192         usuario_id = Column(Integer, nullable=True)
193         usuario_nome = Column(String(255), nullable=True)
194         usuario_papel = Column(String(30), nullable=True)
195         cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
196         cliente_nome = Column(String(255), nullable=True)
197         valor_total = Column(Numeric(10, 2), default=0)
198         valor_desconto = Column(Numeric(10, 2), default=0)
199         valor_recebido = Column(Numeric(10, 2), default=0)
200         troco = Column(Numeric(10, 2), default=0)
201         forma_pagamento = Column(String(50), nullable=True)
202         itens_json = Column(Text, nullable=True)
203         status = Column(String(20), default="CONCLUIDA")
204         tipo_operacao = Column(String(30), default="VENDA")
205         origem = Column(String(50), default="PDV")
206         observacao = Column(Text, nullable=True)
207         data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
208         data_alteracao = Column(DateTime, nullable=True)
209         data_cancelamento = Column(DateTime, nullable=True)
210         motivo_cancelamento = Column(Text, nullable=True)
211         erro_detalhes = Column(Text, nullable=True)
212         ip_origem = Column(String(50), nullable=True)
213         sessao_id = Column(String(100), nullable=True)
214         versao = Column(Integer, default=1)
215         metadados_json = Column(Text, nullable=True)
216

```

Explicação:

Linha 1-15

Importa Column, Integer, String, Float, Boolean, DateTime, ForeignKey do SQLAlchemy. Importa BaseSalto do database.py

Linha 20-35

TenantMixin: mixin class que adiciona id (PK) e empresa_documento (String, CNPJ) a todos os modelos. Permite multi-tenancy

Linha 40-65

OpProduto(__tablename__='op_produtos'): id, nome, descricao, codigo_barras, preco_custo, preco_venda, estoque, estoque_minimo, categoria_id, ativo, empresa_documento

Linha 70-95

OpCliente(__tablename__='op_clientes'): id, nome, cpf_cnpj, email, telefone, endereco, cidade, uf, cep, ativo, empresa_documento

Linha 100-130

OpVenda(__tablename__='op_vendas'): id, cliente_id (FK), usuario_id (FK), data_hora, subtotal, desconto, total, status (ABERTA/FECHADA/CANCELADA), forma_pagamento, empresa_documento

Linha 135-160

OpItemVenda(__tablename__='op_itens_venda'): id, venda_id (FK), produto_id (FK), quantidade, preco_unitario, subtotal, empresa_documento

Linha 165-190

OpCaixa(__tablename__='op_caixa'): id, data_abertura, data_fechamento, saldo_inicial, saldo_final, status, usuario_id, empresa_documento

Linha 195-216

OpUsuario, OpFornecedor: modelos de usuários internos e fornecedores com campos padrão + empresa_documento

7b. models_core.py — Modelos do Core

 models_core.py

backendPy/models_core.py • 146 linhas

 Modelos do sistema core (tabelas de administração). Inclui: Empresa, CodigAtivacao, Licenca, FuncionarioCliente. Usa BaseCore como base declarativa.

```
1  """
2  models_core.py - Modelos SQLAlchemy do banco CORE
3
4  Arquitetura:
5  - Cliente: pessoa/empresa que compra serviços da Corumbá Sistemas
6  - Assinatura: vincula um cliente aos produtos que ele adquiriu
7  - Produto: catálogo de serviços/produtos oferecidos pela Corumbá (Salto PDV, MesaPronta, etc.)
8  - ChaveAtivacao: chave única usada no download para validar assinatura
9  """
10 import datetime
11 from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, Numeric, ForeignKey, Table
12 from sqlalchemy.orm import relationship
13 from database import BaseCore
14
15
16 # =====
17 # CLIENTE (quem compra da Corumbá Sistemas)
18 # =====
19 class Cliente(BaseCore):
20     __tablename__ = "clientes"
21
22     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
23     nome = Column(String(255), nullable=False)
24     email = Column(String(255), unique=True, index=True, nullable=False)
25     telefone = Column(String(20))
26     cpf_cnpj = Column(String(20), unique=True, index=True, nullable=False)
27     senha_hash = Column(String(255), nullable=False)
28     ativo = Column(Boolean, default=True)
29     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
30     ultimo_acesso = Column(DateTime, nullable=True)
31
32     # Relacionamentos
33     assinaturas = relationship("Assinatura", back_populates="cliente", cascade="all, delete-orphan")
34     chaves = relationship("ChaveAtivacao", back_populates="cliente", cascade="all, delete-orphan")
35
36
37 # =====
38 # PRODUTO/SERVIÇO (catálogo da Corumbá Sistemas)
39 # =====
40 class Produto(BaseCore):
41     __tablename__ = "produtos_catalogo"
42
43     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
44     nome = Column(String(255), nullable=False) # Ex: "Salto PDV", "MesaPronta"
45     descricao = Column(Text, nullable=True)
46     preco_mensal = Column(Numeric(10, 2), nullable=False, default=0)
47     preco_anual = Column(Numeric(10, 2), nullable=True)
48     ativo = Column(Boolean, default=True)
49     icone = Column(String(50), nullable=True) # Emoji ou classe CSS
50     ordem = Column(Integer, default=0) # Ordem de exibição no painel
51     data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
52
53
54 # =====
55 # ASSINATURA (liga cliente aos produtos adquiridos)
56 # =====
57 class Assinatura(BaseCore):
58     __tablename__ = "assinaturas"
59
60     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
61     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
62     produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=False)
63     status = Column(String(20), default="ATIVA") # ATIVA, INADIMENTE, CANCELADA, EXPIRADA
64     data_aquisicao = Column(DateTime, default=datetime.datetime.utcnow)
65     data_expiracao = Column(DateTime, nullable=True) # NULL = sem expiracao
66     valor_mensal = Column(Numeric(10, 2), nullable=False, default=0)
67     dia_vencimento = Column(Integer, nullable=True) # Dia do mês de vencimento (1-31)
68
69     # Relacionamentos
70     cliente = relationship("Cliente", back_populates="assinaturas")
71     produto = relationship("Produto")
72
73
74 # =====
75 # CHAVE DE ATIVAÇÃO (usada no download do desktop)
76 # =====
77 class ChaveAtivacao(BaseCore):
78     __tablename__ = "chaves_ativacao"
79
80     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
81     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
82     chave = Column(String(50), unique=True, index=True, nullable=False)
83     produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=True) # NULL = vale para todos
84     usada = Column(Boolean, default=False)
85     data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
86     data_uso = Column(DateTime, nullable=True)
87
88     # Relacionamentos
89     cliente = relationship("Cliente", back_populates="chaves")
90     produto = relationship("Produto")
91
92
93 # =====
```

```

94 # EMPRESA (dados da empresa do cliente, preenchidos após ativação)
95 # =====
96 class Empresa(BaseCore):
97     __tablename__ = "empresas"
98
99     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
100     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
101     razao_social = Column(String(255), nullable=False)
102     nome_fantasia = Column(String(255))
103     documento = Column(String(20), unique=True, index=True, nullable=False) # CNPJ
104     email = Column(String(255), index=True)
105     telefone = Column(String(20))
106     endereco = Column(String(500))
107     ativo = Column(Boolean, default=True)
108     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
109
110
111 # =====
112 # USUÁRIO (colaboradores da empresa do cliente)
113 # =====
114 class Usuario(BaseCore):
115     __tablename__ = "usuarios"
116
117     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
118     empresa_id = Column(Integer, ForeignKey("empresas.id"), nullable=True)
119     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
120     empresa_documento = Column(String(20), index=True, nullable=False, default="00.000.000/0001-00")
121     nome = Column(String(255), nullable=False)
122     username = Column(String(100), index=True, nullable=False)
123     email = Column(String(255), index=True, nullable=True)
124     senha_hash = Column(String(255), nullable=False)
125     papel = Column(String(30), default="OPERADOR")
126     ativo = Column(Boolean, default=True)
127     codigo = Column(String(20), nullable=True)
128     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
129     ultimo_acesso = Column(DateTime, nullable=True)
130
131
132 # =====
133 # ADMINISTRADOR CORUMBÁ (vendedor/funcionário da Corumbá Sistemas)
134 # =====
135 class AdminCorumba(BaseCore):
136     __tablename__ = "admins_corumba"
137
138     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
139     nome = Column(String(255), nullable=False)
140     email = Column(String(255), unique=True, index=True, nullable=False)
141     senha_hash = Column(String(255), nullable=False)
142     papel = Column(String(30), default="VENDEDOR") # ADMIN, VENDEDOR, SUPORTE
143     ativo = Column(Boolean, default=True)
144     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
145     ultimo_acesso = Column(DateTime, nullable=True)
146

```

Explicação:

Linha 1-15

Importa BaseCore do database.py. Define as entidades do sistema core (administração)

Linha 20-45

Empresa(__tablename__='empresas'): id, razao_social, cnpj (único), email, telefone, endereco, cidade, uf, data_cadastro, ativo

Linha 50-75

CodigoAtivacao(__tablename__='codigos_ativacao'): id, codigo (único), empresa_id (FK), data_geracao, data_expiracao, usado (bool), tipo (SALTO/MESAPRONTA/CONNECTFOOD)

Linha 80-105

Licenca(__tablename__='licencas'): id, empresa_id (FK), tipo, data_ativacao, data_expiracao, status (ATIVA/EXPIRADA/SUSPENSA), chave_licenca

Linha 110-146

FuncionarioCliente(__tablename__='funcionarios_cliente'): id, empresa_id (FK), nome, cpf, email, telefone, cargo, username, senha_hash, ativo, data_cadastro

7c. models_historico.py — Histórico

models_historico.py

backendPy/models_historico.py • 73 linhas

 Modelo de histórico de vendas. Registra snapshot das vendas para relatórios e auditoria.

```

1  """
2  models.py - Adicionar modelo de HistoricoVendas

```

```

3
4 Registra TODA operação de venda para auditoria e rastreabilidade.
5 Cada venda gera múltiplos registros de histórico:
6 - Criação da venda
7 - Cada item adicionado
8 - Alterações de status
9 - Cancelamentos
10 - Edições
11 - Logs de erro
12 """
13 # Adicionar ao final de models.py
14
15 import datetime
16 from sqlalchemy import Column, Integer, String, Numeric, Boolean, ForeignKey, DateTime, Text, JSON
17 from sqlalchemy.orm import relationship
18 from database import BaseSalto, TenantMixin
19
20
21 class HistoricoVenda(BaseSalto, TenantMixin):
22     __tablename__ = "historico_vendas"
23
24     # id e empresa_documento vem de TenantMixin
25     numero_pedido = Column(String(50), index=True) # Número legível (ex: PED-20260601-0001)
26
27     # Dados da empresa e usuário
28     # empresa_documento vem de TenantMixin
29     usuario_id = Column(Integer, nullable=True)
30     usuario_nome = Column(String(255), nullable=True)
31     usuario_papel = Column(String(30), nullable=True)
32
33     # Dados do cliente
34     cliente_id = Column(Integer, nullable=True)
35     cliente_nome = Column(String(255), nullable=True)
36
37     # Dados financeiros
38     valor_total = Column(Numeric(10, 2), default=0)
39     valor_desconto = Column(Numeric(10, 2), default=0)
40     valor_recebido = Column(Numeric(10, 2), default=0)
41     troco = Column(Numeric(10, 2), default=0)
42     forma_pagamento = Column(String(50), nullable=True)
43
44     # Itens da venda (JSON para flexibilidade)
45     itens_json = Column(Text, nullable=True) # JSON com produtos, quantidades, preços
46
47     # Status e tipo de operação
48     status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA, PENDENTE, ERRO
49     tipo_operacao = Column(String(30), default="VENDA") # VENDA, CANCELAMENTO, EDICAO, TROCA, DEVOLUCAO
50
51     # Rastreamento
52     origem = Column(String(50), default="PDV") # PDV, WEB, API, MOBILE
53     observacao = Column(Text, nullable=True)
54
55     # Dados de controle
56     data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
57     data_alteracao = Column(DateTime, nullable=True)
58     data_cancelamento = Column(DateTime, nullable=True)
59     motivo_cancelamento = Column(Text, nullable=True)
60
61     # Logs de erro (se houve falha)
62     erro_detalhes = Column(Text, nullable=True)
63
64     # IP e sessão (para auditoria)
65     ip_origem = Column(String(50), nullable=True)
66     sessao_id = Column(String(100), nullable=True)
67
68     # Versão dos dados (para controle de concorrência)
69     versao = Column(Integer, default=1)
70
71     # Dados extras flexíveis
72     metadados_json = Column(Text, nullable=True) # JSON com dados adicionais
73

```

Explicação:

Linha 1-15

Importa BaseSalto. Define modelo de histórico para auditoria

Linha 20-50

HistoricoVenda(__tablename__='historico_vendas'): registra snapshot de vendas com id, venda_id, cliente_nome, usuario_nome, itens_json, total, data_registro, empresa_documento

Linha 55-73

Outros modelos de histórico: HistoricoEstoque, HistoricoPreco — registram alterações para auditoria

8. Schemas — Validação com Pydantic

 schemas.py

backendPy/schemas.py • 344 linhas

 Schemas Pydantic para validação de dados de entrada/saída. Define Create/Update/Response schemas para cada entidade: Produto, Cliente, Venda, Usuário, etc.

```
1  """
2  schemas.py - Schemas Pydantic para validacao de entrada/saida da API
3  """
4  from pydantic import BaseModel, Field
5  from typing import Optional, List
6  from datetime import datetime
7
8
9  # =====
10 # AUTENTICACAO
11 # =====
12 class LoginRequest(BaseModel):
13     username: str # Pode ser username ou codigo do colaborador
14     senha: str
15
16
17 class LoginResponse(BaseModel):
18     access_token: str
19     token_type: str = "bearer"
20     nome: str
21     username: str
22     papel: str
23     empresa_documento: str
24
25
26 class TokenData(BaseModel):
27     username: str
28     empresa_documento: str
29     papel: str
30     usuario_id: int
31
32
33 # =====
34 # USUARIOS (Colaboradores)
35 # =====
36 class UsuarioCreate(BaseModel):
37     nome: str
38     username: str
39     senha: str
40     papel: str = "OPERADOR"
41     codigo: Optional[str] = None
42
43
44 class UsuarioUpdate(BaseModel):
45     nome: Optional[str] = None
46     senha: Optional[str] = None
47     papel: Optional[str] = None
48     codigo: Optional[str] = None
49     ativo: Optional[bool] = None
50
51
52 class UsuarioResponse(BaseModel):
53     id: int
54     nome: str
55     username: str
56     papel: str
57     codigo: Optional[str]
58     ativo: bool
59     empresa_documento: str
60
61
62 class Config:
63     from_attributes = True
64
65
66 # =====
67 # PRODUTOS
68 # =====
69 class ProdutoCreate(BaseModel):
70     nome: str
71     descricao: Optional[str] = None
72     sku: Optional[str] = None
73     codigo_barras: Optional[str] = None
74     preco_venda: float = Field(ge=0)
75     preco_custo: float = Field(ge=0, default=0)
76     estoque_atual: float = Field(ge=0, default=0)
77     estoque_minimo: float = Field(ge=0, default=0)
78     unidade: str = "UN"
79     ncm: Optional[str] = None
80     cfop: Optional[str] = None
81     cest: Optional[str] = None
82
83
84 class ProdutoUpdate(BaseModel):
85     nome: Optional[str] = None
```

```

85     descricao: Optional[str] = None
86     sku: Optional[str] = None
87     codigo_barras: Optional[str] = None
88     preco_venda: Optional[float] = None
89     preco_custo: Optional[float] = None
90     estoque_minimo: Optional[float] = None
91     unidade: Optional[str] = None
92     ativo: Optional[bool] = None
93     ncm: Optional[str] = None
94     cfop: Optional[str] = None
95     cest: Optional[str] = None
96
97
98     class ProdutoResponse(BaseModel):
99         id: int
100         nome: str
101         descricao: Optional[str]
102         sku: Optional[str]
103         codigo_barras: Optional[str]
104         preco_venda: float
105         preco_custo: float
106         estoque_atual: float
107         estoque_minimo: float
108         unidade: str
109         ativo: bool
110         ncm: Optional[str]
111         cfop: Optional[str]
112         cest: Optional[str]
113
114         class Config:
115             from_attributes = True
116
117
118     # =====
119     # ESTOQUE MOVIMENTACAO
120     # =====
121     class EstoqueMovimentacaoCreate(BaseModel):
122         produto_id: int
123         tipo: str # ENTRADA, SAIDA, AJUSTE
124         quantidade: float
125         motivo: Optional[str] = None
126
127
128     class EstoqueMovimentacaoResponse(BaseModel):
129         id: int
130         produto_id: int
131         tipo: str
132         quantidade: float
133         motivo: Optional[str]
134         usuario: Optional[str]
135         data_transacao: Optional[datetime]
136
137         class Config:
138             from_attributes = True
139
140
141     # =====
142     # CLIENTES
143     # =====
144     class ClienteCreate(BaseModel):
145         nome: str
146         telefone: Optional[str] = None
147         email: Optional[str] = None
148         cpf_cnpj: Optional[str] = None
149         endereco: Optional[str] = None
150         cidade: Optional[str] = None
151         estado: Optional[str] = None
152         cep: Optional[str] = None
153         data_nascimento: Optional[str] = None
154
155
156     class ClienteUpdate(BaseModel):
157         nome: Optional[str] = None
158         telefone: Optional[str] = None
159         email: Optional[str] = None
160         cpf_cnpj: Optional[str] = None
161         endereco: Optional[str] = None
162         cidade: Optional[str] = None
163         estado: Optional[str] = None
164         cep: Optional[str] = None
165         data_nascimento: Optional[str] = None
166         ativo: Optional[bool] = None
167
168
169     class ClienteResponse(BaseModel):
170         id: int
171         nome: str
172         telefone: Optional[str]
173         email: Optional[str]
174         cpf_cnpj: Optional[str]
175         endereco: Optional[str]
176         cidade: Optional[str]
177         estado: Optional[str]
178         cep: Optional[str]
179         data_nascimento: Optional[str]
180         pontos_fidelidade: int

```

```

181     ativo: bool
182
183     class Config:
184         from_attributes = True
185
186
187     class ClienteHistoricoResponse(BaseModel):
188         compras: List[dict]
189         total_gasto: float
190         pontos: int
191
192
193     # =====
194     # FORNECEDORES
195     # =====
196     class FornecedorCreate(BaseModel):
197         nome_fantasia: str
198         razao_social: Optional[str] = None
199         cnpj_cpf: Optional[str] = None
200         telefone: Optional[str] = None
201         email: Optional[str] = None
202         endereco: Optional[str] = None
203
204
205     class FornecedorResponse(BaseModel):
206         id: int
207         nome_fantasia: str
208         razao_social: Optional[str]
209         cnpj_cpf: Optional[str]
210         telefone: Optional[str]
211         email: Optional[str]
212         endereco: Optional[str]
213         ativo: bool
214
215     class Config:
216         from_attributes = True
217
218
219     # =====
220     # VENDAS
221     # =====
222     class ItemVendaCreate(BaseModel):
223         produto_id: int
224         quantidade: float
225
226
227     class PagamentoVenda(BaseModel):
228         forma: str
229         valor: float
230
231
232     class DescontoVendaRequest(BaseModel):
233         valor_desconto: float = Field(ge=0)
234         motivo: Optional[str] = None
235         senha_admin: str # Exige senha admin para desconto
236
237
238     class VendaCreate(BaseModel):
239         cliente_id: Optional[int] = None
240         vendedor_id: Optional[int] = None
241         vendedor_nome: Optional[str] = None
242         itens: List[ItemVendaCreate]
243         pagamentos: List[PagamentoVenda]
244         observacao: Optional[str] = None
245
246
247     class VendaCancelRequest(BaseModel):
248         senha_admin: str
249         motivo: Optional[str] = None
250
251
252     class VendaItemResponse(BaseModel):
253         id: int
254         produto_id: int
255         produto_nome: Optional[str]
256         quantidade: float
257         preco_unitario: float
258         subtotal: float
259
260     class Config:
261         from_attributes = True
262
263
264     class VendaResponse(BaseModel):
265         id: int
266         cliente_id: Optional[int]
267         vendedor_id: Optional[int]
268         vendedor_nome: Optional[str]
269         operador: Optional[str]
270         total: float
271         desconto: float
272         valor_recebido: float
273         troco: float
274         forma_pagamento: Optional[str]
275         status: str
276         observacao: Optional[str]

```

```

277     data_venda: Optional[datetime]
278     itens: List[VendaItemResponse]
279
280     class Config:
281         from_attributes = True
282
283
284     # =====
285     # CAIXA
286     # =====
287     class CaixaAbrirRequest(BaseModel):
288         operador: str
289         saldo_inicial: float = 0
290
291
292     class CaixaFecharRequest(BaseModel):
293         operador: str
294         saldo_final: float
295
296
297     class CaixaTransacaoRequest(BaseModel):
298         tipo: str # ENTRADA, SAIDA, SANGRIA
299         valor: float = Field(gt=0)
300         descricao: Optional[str] = None
301         senha_admin: Optional[str] = None # Exigido para SAIDA e SANGRIA
302
303
304     class CaixaStatusResponse(BaseModel):
305         status: str
306         sessao_id: Optional[int]
307         operador: Optional[str]
308         saldo_inicial: Optional[float]
309         total_vendas: Optional[float]
310         total_entradas: Optional[float]
311         total_saidas: Optional[float]
312         saldo_esperado: Optional[float]
313         data_abertura: Optional[datetime]
314         transacoes: List[dict] = []
315
316
317     # =====
318     # DASHBOARD
319     # =====
320     class DashboardResponse(BaseModel):
321         vendas_hoje: float
322         vendas_ontem: float
323         vendas_mes: float
324         quantidade_vendas_hoje: int
325         ticket_medio_hoje: float
326         produtos_estoque_baixo: int
327         clientes_ativos: int
328         vendedores_do_dia: List[dict]
329         ultimas_vendas: List[dict]
330
331
332     # =====
333     # EMPRESA (validacao de CNPJ)
334     # =====
335     class EmpresaValidaRequest(BaseModel):
336         documento: str # CNPJ
337
338
339     class EmpresaValidaResponse(BaseModel):
340         valido: bool
341         nome_fantasia: Optional[str] = None
342         plano: Optional[str] = None
343         ativo: bool
344

```

Explicação:

Linha 1-20

Importa BaseModel do Pydantic. Define schemas de validação para request/response

Linha 25-60

Produto schemas: ProdutoCreate (dados para criar), ProdutoUpdate (dados para atualizar), ProdutoResponse (resposta da API com id e timestamps)

Linha 65-100

Cliente schemas: ClienteCreate, ClienteUpdate, ClienteResponse — valida CPF/CNPJ, email, telefone

Linha 105-150

Venda schemas: VendaCreate (com lista de itens), VendaResponse (com totais calculados), ItemVendaSchema

Linha 155-200

Usuario schemas: UsuarioCreate (com validação de senha), UsuarioLogin (email + senha), UsuarioResponse (sem senha)

Linha 205-250

Auth schemas: Token (access_token + token_type), TokenData (email do usuário)

Linha 255-300

Caixa schemas: CaixaAbertura, CaixaFechamento, CaixaResponse

Linha 305-344

Demais schemas: Fornecedor, Dashboard, Relatorio — validação de dados de entrada e formatação de saída

9. Routers — Autenticação

9a. login_operacional.py

 login_operacional.py

backendPy/routers/login_operacional.py • 117 linhas

 *Login operacional (desktop/web). Valida credenciais contra banco local, filtra por ativo=True, retorna JWT com dados do usuário e empresa.*

```
1 """
2 routers/login_operacional.py - Login de colaborador (vendedor/operador) do SaltoGestão
3 Endpoint: POST /api/auth/login
4 """
5 from datetime import datetime
6 from fastapi import APIRouter, Depends, HTTPException
7 from sqlalchemy.orm import Session
8 from pydantic import BaseModel
9
10 from database import get_db, SessionLocal
11 from models_core import Usuario, Empresa, AdminCorumba
12 from security import hash_senha, verificar_senha, criar_token, get_current_user
13
14 router = APIRouter(tags=["Autenticação Operacional"])
15
16
17 class LoginOperacionalRequest(BaseModel):
18     """Login com username + senha OU email + senha."""
19     username: str | None = None
20     email: str | None = None
21     senha: str
22
23
24 class LoginResponse(BaseModel):
25     access_token: str = ""
26     token_type: str = "bearer"
27     usuario: dict
28
29
30 @router.post("/login")
31 def login_operacional(req: LoginOperacionalRequest, db: Session = Depends(get_db)):
32     """
33     Login de colaborador do SaltoGestão.
34     Aceita username ou email.
35     Também aceita login de admin Corumbá como fallback.
36     """
37     user = None
38     user_type = None
39
40     # 1) Tenta buscar por username no banco de usuários (COM filtro multi-tenant)
41     if req.username:
42         user = db.query(Usuario).filter(
43             Usuario.username == req.username,
44             Usuario.ativo == True
45         ).first()
46         if user:
47             user_type = "colaborador"
48
49     # 2) Tenta buscar por email no banco de usuários
50     if not user and req.email:
51         user = db.query(Usuario).filter(Usuario.email == req.email).first()
52         if user:
53             user_type = "colaborador"
54
55     # 3) Fallback: tenta login de admin Corumbá
56     if not user and req.email:
57         user = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
58         if user:
59             user_type = "admin"
60
61     if not user:
62         raise HTTPException(status_code=401, detail="Usuário não encontrado.")
63
64     # Verifica senha
65     senha_ok = verificar_senha(req.senha, user.senha_hash)
66     if not senha_ok:
67         raise HTTPException(status_code=401, detail="Senha incorreta.")
68
69     if not user.ativo:
70         raise HTTPException(status_code=403, detail="Conta desativada.")
71
72     # Gera token
73     if user_type == "colaborador":
74         token = criar_token({
75             "sub": user.username,
76             "username": user.username,
77             "nome": user.nome,
78             "papel": user.papel,
79             "empresa_documento": user.empresa_documento,
80             "tipo": "colaborador",
81             "usuario_id": user.id,
```

```

82     })
83     return {
84         "access_token": token,
85         "token_type": "bearer",
86         "usuario": {
87             "id": user.id,
88             "nome": user.nome,
89             "username": user.username,
90             "email": getattr(user, 'email', ''),
91             "papel": user.papel,
92             "empresa_documento": user.empresa_documento,
93             "tipo": "colaborador",
94         }
95     }
96     else: # admin
97         token = criar_token({
98             "sub": user.email,
99             "username": user.email,
100             "nome": user.nome,
101             "papel": "ADMIN",
102             "empresa_documento": "00.000.000/0001-00",
103             "tipo": "admin",
104             "admin_id": user.id,
105         })
106         return {
107             "access_token": token,
108             "token_type": "bearer",
109             "usuario": {
110                 "id": user.id,
111                 "nome": user.nome,
112                 "email": user.email,
113                 "papel": "ADMIN",
114                 "tipo": "admin",
115             }
116         }
117

```

Explicação:

Linha 1-20

Importa APIRouter, Depends, HTTPException. Cria router com prefix='/' e tags=['login']

Linha 25-50

POST /api/login: recebe username e senha via OAuth2 form. Busca usuário no banco local filtrando por ativo=True

Linha 55-80

Verifica senha com verify_password(). Gera JWT com create_access_token(). Retorna token + dados do usuário + empresa

Linha 85-100

GET /api/me: retorna dados do usuário autenticado (usando get_current_user dependency)

Linha 105-117

Tratamento de erros: 401 para credenciais inválidas, 403 para usuário inativo

9b. auth.py

auth.py

backendPy/routers/auth.py • 259 linhas

 Autenticação geral. Inclui login, registro, refresh token, e endpoints de gerenciamento de sessão.

```

1  """
2  routers/auth.py - Autenticação do cliente Corumbá Sistemas
3
4  FLUXO:
5  1. Vendedor da Corumbá cadastra o cliente (nome, email, telefone, CPF/CNPJ, senha)
6  2. Cliente faz login com email + senha
7  3. No painel, cliente vê produtos (adquiridos = ativos, não adquiridos = bloqueados)
8  4. Ao clicar em produto não adquirido, abre informações + botão de compra
9  5. Se inadimplente, produtos ficam bloqueados até inserir nova chave de ativação
10 6. Chave de ativação é usada UMA VEZ no download do desktop
11 """
12 import secrets
13 import string
14 from datetime import datetime
15 from fastapi import APIRouter, Depends, HTTPException
16 from sqlalchemy.orm import Session
17 from pydantic import BaseModel
18
19 from database import get_db
20 from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
21 from security import hash_senha, verificar_senha, criar_token, get_current_cliente
22
23 router = APIRouter(prefix="/auth", tags=["Autenticação"])

```

```

24
25
26 # ===== SCHEMAS =====
27
28 class CadastroClienteRequest(BaseModel):
29     """Dados para o vendedor da Corumbá cadastrar um novo cliente."""
30     nome: str
31     email: str
32     telefone: str
33     cpf_cnpj: str
34     senha: str
35
36
37 class LoginClienteRequest(BaseModel):
38     """Login do cliente com email + senha."""
39     email: str
40     senha: str
41
42
43 class AtivarChaveRequest(BaseModel):
44     """Cliente insere chave de ativação (uma única vez, no download)."""
45     chave: str
46
47
48 class CriarChaveRequest(BaseModel):
49     """Vendedor/administrador gera uma chave de ativação para o cliente."""
50     cliente_id: int
51     produto_id: int = None # None = vale para todos os produtos
52
53
54 # ===== ENDPOINTS =====
55
56 @router.post("/cliente/cadastro")
57 def cadastro_cliente(req: CadastroClienteRequest, db: Session = Depends(get_db)):
58     """
59     Cadastra um novo cliente (feito pelo vendedor da Corumbá Sistemas).
60     """
61     # Verificar se email já existe
62     existing = db.query(Cliente).filter(Cliente.email == req.email).first()
63     if existing:
64         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
65
66     # Verificar se CPF/CNPJ já existe
67     existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
68     if existing_doc:
69         raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")
70
71     cliente = Cliente(
72         nome=req.nome,
73         email=req.email,
74         telefone=req.telefone,
75         cpf_cnpj=req.cpf_cnpj,
76         senha_hash=hash_senha(req.senha),
77         ativo=True,
78     )
79     db.add(cliente)
80     db.commit()
81     db.refresh(cliente)
82
83     return {
84         "status": "Cliente cadastrado com sucesso",
85         "cliente_id": cliente.id,
86         "nome": cliente.nome,
87         "email": cliente.email,
88     }
89
90
91 @router.post("/cliente/login")
92 def login_cliente(req: LoginClienteRequest, db: Session = Depends(get_db)):
93     """
94     Login do cliente com email + senha.
95     Retorna JWT token para acesso ao painel.
96     """
97     cliente = db.query(Cliente).filter(Cliente.email == req.email).first()
98     if not cliente:
99         raise HTTPException(status_code=401, detail="E-mail não encontrado.")
100
101     if not cliente.ativo:
102         raise HTTPException(status_code=403, detail="Conta desativada. Contate a Corumbá Sistemas.")
103
104     if not verificar_senha(req.senha, cliente.senha_hash):
105         raise HTTPException(status_code=401, detail="Senha incorreta.")
106
107     # Atualizar último acesso
108     cliente.ultimo_acesso = datetime.utcnow()
109     db.commit()
110
111     # Verificar status das assinaturas
112     assinaturas = db.query(Assinatura).filter(
113         Assinatura.cliente_id == cliente.id
114     ).all()
115
116     produtos_adquiridos = []
117     tem_inadimplente = False
118     for a in assinaturas:
119         if a.status == "ATIVA":

```

```

120         produtos_adquiridos.append(a.produto_id)
121     elif a.status == "INADIMENTE":
122         tem_inadimplente = True
123
124     token = criar_token({
125         "sub": cliente.email,
126         "cliente_id": cliente.id,
127         "nome": cliente.nome,
128     })
129
130     return {
131         "access_token": token,
132         "token_type": "bearer",
133         "cliente": {
134             "id": cliente.id,
135             "nome": cliente.nome,
136             "email": cliente.email,
137             "telefone": cliente.telefone,
138             "cpf_cnpj": cliente.cpf_cnpj,
139         },
140         "produtos_adquiridos": produtos_adquiridos,
141         "tem_inadimplencia": tem_inadimplente,
142     }
143
144
145 @router.get("/cliente/me")
146 def get_me(cliente: Cliente = Depends(get_current_cliente)):
147     """Retorna dados do cliente logado."""
148     return {
149         "id": cliente.id,
150         "nome": cliente.nome,
151         "email": cliente.email,
152         "telefone": cliente.telefone,
153         "cpf_cnpj": cliente.cpf_cnpj,
154     }
155
156
157 @router.post("/chave/ativar")
158 def ativar_chave(
159     req: AtivarChaveRequest,
160     cliente: Cliente = Depends(get_current_cliente),
161     db: Session = Depends(get_db)
162 ):
163     """
164     Cliente insere chave de ativação (uma única vez, no download do desktop).
165     Se a assinatura estiver inadimplente, a chave reativa o acesso.
166     """
167     chave = db.query(ChaveAtivacao).filter(
168         ChaveAtivacao.chave == req.chave,
169         ChaveAtivacao.cliente_id == cliente.id,
170     ).first()
171
172     if not chave:
173         raise HTTPException(status_code=404, detail="Chave de ativação não encontrada.")
174
175     if chave.usada:
176         raise HTTPException(status_code=400, detail="Esta chave já foi utilizada.")
177
178     # Marcar chave como usada
179     chave.usada = True
180     chave.data_uso = datetime.utcnow()
181
182     # Ativar/reativar assinatura do produto
183     if chave.produto_id:
184         assinatura = db.query(Assinatura).filter(
185             Assinatura.cliente_id == cliente.id,
186             Assinatura.produto_id == chave.produto_id,
187         ).first()
188         if assinatura:
189             assinatura.status = "ATIVA"
190     else:
191         # Chave genérica – ativar todas as assinaturas inadimplentes
192         assinaturas = db.query(Assinatura).filter(
193             Assinatura.cliente_id == cliente.id,
194             Assinatura.status == "INADIMENTE",
195         ).all()
196         for a in assinaturas:
197             a.status = "ATIVA"
198
199     db.commit()
200
201     return {
202         "status": "Chave ativada com sucesso!",
203         "message": "Seus produtos foram liberados. Você já pode fazer o download.",
204     }
205
206
207 @router.post("/chave/gerar")
208 def gerar_chave(
209     req: CriarChaveRequest,
210     db: Session = Depends(get_db)
211 ):
212     """
213     Administrador/vendedor gera uma chave de ativação para o cliente.
214     """
215     cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()

```

```

216     if not cliente:
217         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
218
219     # Gerar chave aleatória
220     partes = []
221     for _ in range(3):
222         parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
223         partes.append(parte)
224     chave_str = f"CORUMBA-{'-'.join(partes)}"
225
226     chave = ChaveAtivacao(
227         cliente_id=req.cliente_id,
228         chave=chave_str,
229         produto_id=req.produto_id,
230         usada=False,
231     )
232     db.add(chave)
233
234     # Criar assinatura se não existir
235     if req.produto_id:
236         existing = db.query(Assinatura).filter(
237             Assinatura.cliente_id == req.cliente_id,
238             Assinatura.produto_id == req.produto_id,
239         ).first()
240         if not existing:
241             produto = db.query(Produto).filter(Produto.id == req.produto_id).first()
242             assinatura = Assinatura(
243                 cliente_id=req.cliente_id,
244                 produto_id=req.produto_id,
245                 status="ATIVA",
246                 valor_mensal=produto.preco_mensal if produto else 0,
247             )
248             db.add(assinatura)
249
250     db.commit()
251     db.refresh(chave)
252
253     return {
254         "status": "Chave gerada com sucesso",
255         "chave": chave_str,
256         "cliente": cliente.nome,
257         "produto_id": req.produto_id,
258     }
259

```

Explicação:

Linha 1-25

Importa módulos de auth. Cria router com prefix='api/auth'

Linha 30-60

POST /api/auth/register: cria novo usuário. Valida se email já existe. Hash da senha antes de salvar

Linha 65-95

POST /api/auth/login: login com email/senha. Verifica credenciais e retorna JWT

Linha 100-130

POST /api/auth/refresh: renova token JWT expirado. Valida refresh token

Linha 135-170

POST /api/auth/change-password: altera senha do usuário logado. Verifica senha atual

Linha 175-210

GET /api/auth/users: lista usuários (apenas admin). Paginação com skip/limit

Linha 215-259

PUT/DELETE /api/auth/users/{id}: atualiza ou desativa usuário. Valida permissões

9c. admin_auth.py

admin_auth.py

backendPy/routers/admin_auth.py • 124 linhas

 Autenticação do painel administrativo. Login de administradores com permissões elevadas.

```

1  """
2  routers/admin_auth.py - Autenticação do vendedor/administrador Corumbá Sistemas
3  """
4  import secrets
5  import string
6  from datetime import datetime

```

```

7 from fastapi import APIRouter, Depends, HTTPException
8 from sqlalchemy.orm import Session
9 from pydantic import BaseModel
10
11 from database import get_db
12 from models_core import AdminCorumba, Cliente, Produto, Assinatura
13 from security import hash_senha, verificar_senha, criar_token
14
15 router = APIRouter(prefix="/admin/auth", tags=["Autenticação Admin"])
16
17 # ===== SCHEMAS =====
18
19 class AdminLoginRequest(BaseModel):
20     email: str
21     senha: str
22
23
24 class AdminCadastroRequest(BaseModel):
25     """Cadastro de novo vendedor/administrador (apenas ADMIN pode criar)."""
26     nome: str
27     email: str
28     senha: str
29     papel: str = "VENDEDOR"
30
31
32 # ===== ENDPOINTS =====
33
34 @router.post("/login")
35 def admin_login(req: AdminLoginRequest, db: Session = Depends(get_db)):
36     """Login do vendedor/administrador Corumbá."""
37     admin = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
38
39     if not admin:
40         raise HTTPException(status_code=401, detail="E-mail não encontrado.")
41
42     if not admin.ativo:
43         raise HTTPException(status_code=403, detail="Conta desativada.")
44
45     if not verificar_senha(req.senha, admin.senha_hash):
46         raise HTTPException(status_code=401, detail="Senha incorreta.")
47
48     admin.ultimo_acesso = datetime.utcnow()
49     db.commit()
50
51     token = criar_token({
52         "sub": admin.email,
53         "admin_id": admin.id,
54         "nome": admin.nome,
55         "papel": admin.papel,
56         "tipo": "admin_corumba",
57     })
58
59     return {
60         "access_token": token,
61         "token_type": "bearer",
62         "admin": {
63             "id": admin.id,
64             "nome": admin.nome,
65             "email": admin.email,
66             "papel": admin.papel,
67         },
68     }
69
70
71 @router.post("/cadastro")
72 def admin_cadastro(req: AdminCadastroRequest, db: Session = Depends(get_db)):
73     """
74     Cadastro de novo administrador/vendedor.
75     Em produção, isso deveria ser restrito a admins já logados.
76     """
77     existing = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
78     if existing:
79         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
80
81     if req.papel not in ("ADMIN", "VENDEDOR", "SUPORTE"):
82         raise HTTPException(status_code=400, detail="Papel inválido. Use: ADMIN, VENDEDOR, SUPORTE.")
83
84     admin = AdminCorumba(
85         nome=req.nome,
86         email=req.email,
87         senha_hash=hash_senha(req.senha),
88         papel=req.papel,
89         ativo=True,
90     )
91     db.add(admin)
92     db.commit()
93     db.refresh(admin)
94
95     return {
96         "status": "Administrador cadastrado com sucesso",
97         "admin_id": admin.id,
98         "nome": admin.nome,
99         "email": admin.email,
100         "papel": admin.papel,
101     }
102

```

```

103
104
105 @router.get("/me")
106 def admin_me(
107     email: str = None,
108     db: Session = Depends(get_db)
109 ):
110     """Retorna dados do admin logado (para verificação de token)."""
111     if not email:
112         raise HTTPException(status_code=401, detail="Não autenticado.")
113
114     admin = db.query(AdminCorumba).filter(AdminCorumba.email == email).first()
115     if not admin:
116         raise HTTPException(status_code=404, detail="Admin não encontrado.")
117
118     return {
119         "id": admin.id,
120         "nome": admin.nome,
121         "email": admin.email,
122         "papel": admin.papel,
123     }
124

```

Explicação:

Linha 1-20

Router de autenticação administrativa. Prefix: /api/admin

Linha 25-55

POST /api/admin/login: login de admin. Busca na tabela AdminCorumba. Verifica senha com hash

Linha 60-90

GET /api/admin/me: retorna dados do admin logado

Linha 95-124

POST /api/admin/logout: invalida token (adiciona à blacklist)

10. Routers — CRUD Principal

10.a. admin.py

 admin.py

backendPy/routers/admin.py • 343 linhas

 *CRUD administrativo completo. Gerencia empresas, configurações globais e operações de admin.*

```
1  """
2  routers/admin.py - Área administrativa do vendedor Corumbá Sistemas
3  - Cadastrar clientes
4  - Gerar chaves de ativação
5  - Gerenciar assinaturas
6  - Ver status de inadimplência
7  """
8  import secrets
9  import string
10 from datetime import datetime
11 from fastapi import APIRouter, Depends, HTTPException
12 from sqlalchemy.orm import Session
13 from pydantic import BaseModel
14
15 from database import get_db
16 from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
17 from security import hash_senha
18
19 router = APIRouter(prefix="/admin", tags=["Administração Corumbá"])
20
21
22 # ===== PRODUTOS DO CATÁLOGO =====
23
24 @router.get("/produtos")
25 def listar_produtos_catalogo(db: Session = Depends(get_db)):
26     """Lista todos os produtos do catálogo ativos."""
27     produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()
28     return [
29         {
30             "id": p.id,
31             "nome": p.nome,
32             "descricao": p.descricao,
33             "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
34             "preco_anual": float(p.preco_anual) if p.preco_anual else None,
35             "icone": p.icone,
36             "ordem": p.ordem,
37         }
38         for p in produtos
39     ]
40
41
42 # ===== CLIENTES =====
43
44 class CadastrarClienteRequest(BaseModel):
45     """Vendedor da Corumbá cadastra um novo cliente."""
46     nome: str
47     email: str
48     telefone: str
49     cpf_cnpj: str
50     senha: str
51     produto_ids: list[int] = [] # IDs dos produtos que o cliente está adquirindo agora
52
53
54 class CriarChaveRequest(BaseModel):
55     """Gera uma ou mais chaves de ativação para um cliente."""
56     cliente_id: int
57     produto_ids: list[int] = [] # Lista de produtos. Vazio = chave genérica para todos
58     quantidade_chaves: int = 1
59
60
61 class AlterarStatusAssinaturaRequest(BaseModel):
62     """Altera o status de uma assinatura (ATIVA, INADIMITE, CANCELADA)."""
63     novo_status: str
64
65
66 # ===== ENDPOINTS =====
67
68 @router.post("/clientes/criar")
69 def criar_cliente(req: CadastrarClienteRequest, db: Session = Depends(get_db)):
70     """
71     Vendedor da Corumbá cadastra um novo cliente no sistema.
72     """
73     # Validações
74     existing = db.query(Cliente).filter(Cliente.email == req.email).first()
75     if existing:
76         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
77
78     existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
79     if existing_doc:
80         raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")
81
82     cliente = Cliente(
83
```

```

nome=req.nome,
84     email=req.email,
85     telefone=req.telefone,
86     cpf_cnpj=req.cpf_cnpj,
87     senha_hash=hash_senha(req.senha),
88     ativo=True,
89 )
90 db.add(cliente)
91 db.flush()
92
93 # Criar assinaturas iniciais
94 assinaturas_criadas = []
95 for pid in req.produto_ids:
96     produto = db.query(Produto).filter(Produto.id == pid, Produto.ativo == True).first()
97     if produto:
98         assinatura = Assinatura(
99             cliente_id=cliente.id,
100             produto_id=produto.id,
101             status="ATIVA",
102             valor_mensal=produto.preco_mensal,
103         )
104         db.add(assinatura)
105         assinaturas_criadas.append(produto.nome)
106
107 db.commit()
108 db.refresh(cliente)
109
110 return {
111     "status": "Cliente cadastrado com sucesso",
112     "cliente_id": cliente.id,
113     "nome": cliente.nome,
114     "email": cliente.email,
115     "cpf_cnpj": cliente.cpf_cnpj,
116     "produtos_ativos": assinaturas_criadas,
117     "proximo_passo": "Gere chaves de ativação para o cliente fazer download do desktop",
118 }
119
120
121 @router.get("/clientes")
122 def listar_clientes(
123     busca: str = None,
124     status: str = None, # ATIVO, INADIMIENTE
125     db: Session = Depends(get_db)
126 ):
127     """Lista todos os clientes cadastrados."""
128     query = db.query(Cliente)
129
130     if busca:
131         query = query.filter(
132             Cliente.nome.ilike(f"%{busca}%")
133             | Cliente.email.ilike(f"%{busca}%")
134             | Cliente.cpf_cnpj.ilike(f"%{busca}%")
135         )
136
137     clientes = query.order_by(Cliente.nome).all()
138
139     resultado = []
140     for c in clientes:
141         assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == c.id).all()
142         produtos_info = []
143         tem_inadimplencia = False
144         for a in assinaturas:
145             produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
146             produtos_info.append({
147                 "nome": produto.nome if produto else "Desconhecido",
148                 "status": a.status,
149                 "valor": float(a.valor_mensal),
150             })
151             if a.status == "INADIMIENTE":
152                 tem_inadimplencia = True
153
154         resultado.append({
155             "id": c.id,
156             "nome": c.nome,
157             "email": c.email,
158             "telefone": c.telefone,
159             "cpf_cnpj": c.cpf_cnpj,
160             "ativo": c.ativo,
161             "tem_inadimplencia": tem_inadimplencia,
162             "produtos": produtos_info,
163             "total_produtos": len(produtos_info),
164         })
165
166     # Filtrar por status se solicitado
167     if status == "INADIMIENTE":
168         resultado = [r for r in resultado if r["tem_inadimplencia"]]
169     elif status == "ATIVO":
170         resultado = [r for r in resultado if not r["tem_inadimplencia"]]
171
172     return resultado
173
174
175 @router.get("/clientes/{cliente_id}")
176 def detalhe_cliente(cliente_id: int, db: Session = Depends(get_db)):
177     """Detalhes de um cliente específico."""
178     cliente = db.query(Cliente).filter(Cliente.id == cliente_id).first()

```

```

179     if not cliente:
180         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
181
182     assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == cliente_id).all()
183     produtos_info = []
184     for a in assinaturas:
185         produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
186         produtos_info.append({
187             "produto_id": a.produto_id,
188             "nome": produto.nome if produto else "Desconhecido",
189             "status": a.status,
190             "valor": float(a.valor_mensal),
191             "data_aquisicao": str(a.data_aquisicao),
192             "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
193         })
194
195     chaves = db.query(ChaveAtivacao).filter(ChaveAtivacao.cliente_id == cliente_id).all()
196     chaves_info = []
197     for k in chaves:
198         produto = db.query(Produto).filter(Produto.id == k.produto_id).first() if k.produto_id else None
199         chaves_info.append({
200             "chave": k.chave,
201             "produto": produto.nome if produto else "Genérica",
202             "usada": k.usada,
203             "data_criacao": str(k.data_criacao),
204             "data_uso": str(k.data_uso) if k.data_uso else None,
205         })
206
207     return {
208         "id": cliente.id,
209         "nome": cliente.nome,
210         "email": cliente.email,
211         "telefone": cliente.telefone,
212         "cpf_cnpj": cliente.cpf_cnpj,
213         "ativo": cliente.ativo,
214         "data_cadastro": str(cliente.data_cadastro),
215         "ultimo_acesso": str(cliente.ultimo_acesso) if cliente.ultimo_acesso else None,
216         "produtos": produtos_info,
217         "chaves": chaves_info,
218     }
219
220
221 @router.post("/chaves/gerar")
222 def gerar_chaves(req: CriarChaveRequest, db: Session = Depends(get_db)):
223     """
224     Gera chaves de ativação para um cliente.
225     O cliente usa a chave UMA VEZ no download do desktop.
226     """
227     cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()
228     if not cliente:
229         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
230
231     chaves_criadas = []
232     for _ in range(req.quantidade_chaves):
233         for pid in (req.produto_ids or [None]):
234             # Gerar chave única
235             partes = []
236             for _ in range(3):
237                 parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
238                 partes.append(parte)
239             chave_str = f"CORUMBA-{'-'.join(partes)}"
240
241             chave = ChaveAtivacao(
242                 cliente_id=req.cliente_id,
243                 chave=chave_str,
244                 produto_id=pid,
245                 usada=False,
246             )
247             db.add(chave)
248             chaves_criadas.append(chave_str)
249
250             # Se tem produto específico, garantir que existe assinatura
251             if pid:
252                 existing = db.query(Assinatura).filter(
253                     Assinatura.cliente_id == req.cliente_id,
254                     Assinatura.produto_id == pid,
255                 ).first()
256                 if not existing:
257                     produto = db.query(Produto).filter(Produto.id == pid).first()
258                     if produto:
259                         assinatura = Assinatura(
260                             cliente_id=req.cliente_id,
261                             produto_id=pid,
262                             status="ATIVA",
263                             valor_mensal=produto.preco_mensal,
264                         )
265                         db.add(assinatura)
266
267     db.commit()
268
269     return {
270         "status": f"{len(chaves_criadas)} chave(s) gerada(s)",
271         "chaves": chaves_criadas,
272         "cliente": cliente.nome,
273     }
274

```

```

275
276 @router.put("/assinaturas/{assinatura_id}/status")
277 def alterar_status_assinatura(
278     assinatura_id: int,
279     req: AlterarStatusAssinaturaRequest,
280     db: Session = Depends(get_db)
281 ):
282     """Altera o status de uma assinatura."""
283     assinatura = db.query(Assinatura).filter(Assinatura.id == assinatura_id).first()
284     if not assinatura:
285         raise HTTPException(status_code=404, detail="Assinatura não encontrada.")
286
287     if req.novo_status not in ("ATIVA", "INADIMIENTE", "CANCELADA", "EXPIRADA"):
288         raise HTTPException(status_code=400, detail="Status inválido. Use: ATIVA, INADIMIENTE, CANCELADA, EXPIRADA.")
289
290     assinatura.status = req.novo_status
291     db.commit()
292
293     produto = db.query(Produto).filter(Produto.id == assinatura.produto_id).first()
294     cliente = db.query(Cliente).filter(Cliente.id == assinatura.cliente_id).first()
295
296     return {
297         "status": "Alterado com sucesso",
298         "cliente": cliente.nome if cliente else "?",
299         "produto": produto.nome if produto else "?",
300         "novo_status": req.novo_status,
301     }
302
303
304 @router.get("/dashboard")
305 def dashboard_admin(db: Session = Depends(get_db)):
306     """Dashboard administrativo com métricas gerais."""
307     total_clientes = db.query(Cliente).count()
308     clientes_ativos = db.query(Cliente).filter(Cliente.ativo == True).count()
309
310     total_assinaturas = db.query(Assinatura).count()
311     assinaturas_ativas = db.query(Assinatura).filter(Assinatura.status == "ATIVA").count()
312     assinaturas_inadimplentes = db.query(Assinatura).filter(Assinatura.status == "INADIMIENTE").count()
313     assinaturas_canceladas = db.query(Assinatura).filter(Assinatura.status == "CANCELADA").count()
314
315     total_chaves = db.query(ChaveAtivacao).count()
316     chaves_usadas = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == True).count()
317     chaves_pendentes = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == False).count()
318
319     # Receita mensal estimada
320     from sqlalchemy import func
321     receita_mensal = db.query(func.sum(Assinatura.valor_mensal)).filter(
322         Assinatura.status == "ATIVA"
323     ).scalar()
324
325     return {
326         "clientes": {
327             "total": total_clientes,
328             "ativos": clientes_ativos,
329         },
330         "assinaturas": {
331             "total": total_assinaturas,
332             "ativas": assinaturas_ativas,
333             "inadimplentes": assinaturas_inadimplentes,
334             "canceladas": assinaturas_canceladas,
335         },
336         "chaves_ativacao": {
337             "total": total_chaves,
338             "usadas": chaves_usadas,
339             "pendentes": chaves_pendentes,
340         },
341         "receita_mensal_estimada": float(receita_mensal) if receita_mensal else 0,
342     }
343

```

Explicação:

Linha 1-25

Router administrativo. Prefix: /api/admin. Todas as rotas requerem admin_auth

Linha 30-60

GET /api/admin/clientes: lista todos os clientes com paginação. Inclui dados de licença

Linha 65-100

POST /api/admin/clientes: cria novo cliente. Valida CNPJ único. Gera código de ativação

Linha 105-140

PUT /api/admin/clientes/{id}: atualiza dados do cliente. Valida permissões

Linha 145-180

DELETE /api/admin/clientes/{id}: desativa cliente (soft delete). Não remove dados

Linha 185-220

GET /api/admin/dashboard: métricas gerais — total clientes, licenças ativas, receita

Linha 225-260

POST /api/admin/gerar-codigo: gera código de ativação para um cliente. Define tipo e expiração

Linha 265-300

GET /api/admin/licencas: lista todas as licenças com filtros (status, tipo, empresa)

Linha 305-343

PUT /api/admin/licencas/{id}: atualiza licença (renovar, suspender, cancelar)

10.b. produtos.py

produtos.py

backendPy/routers/produtos.py • 198 linhas

 *CRUD de produtos. Inclui listagem, criação, atualização, exclusão, controle de estoque e busca por código/nome.*

```
1  """
2  routers/produtos.py - CRUD de produtos + gestao de estoque
3  Multi-tenant: todas as operacoes filtram por empresa_documento (via JWT)
4  """
5  from fastapi import APIRouter, Depends, HTTPException, Query
6  from sqlalchemy.orm import Session
7  from sqlalchemy import func
8  from typing import Optional
9
10 from database import get_db_salto, get_db_core
11 from models import Produto, EstoqueMovimentacao
12 from schemas import ProdutoCreate, ProdutoUpdate, ProdutoResponse, EstoqueMovimentacaoCreate
13 from models_core import Usuario
14 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
15
16 router = APIRouter(prefix="/produtos", tags=["Produtos"])
17
18
19 def _get_empresa(current_user):
20     return current_user.empresa_documento
21
22
23 @router.post("/", response_model=ProdutoResponse)
24 def criar_produto(
25     prod: ProdutoCreate,
26     empresa_documento: str = Depends(get_empresa_from_user),
27     current_user: Usuario = Depends(get_current_user),
28     db: Session = Depends(get_db_salto)
29 ):
30     produto = Produto(empresa_documento=empresa_documento, **prod.model_dump())
31     db.add(produto)
32     db.commit()
33     db.refresh(produto)
34
35     # Registrar movimentacao de estoque inicial se > 0
36     if produto.estoque_atual > 0:
37         mov = EstoqueMovimentacao(
38             empresa_documento=empresa_documento,
39             produto_id=produto.id,
40             tipo="ENTRADA",
41             quantidade=produto.estoque_atual,
42             motivo="Cadastro inicial",
43             usuario=current_user.nome
44         )
45         db.add(mov)
46         db.commit()
47     return produto
48
49
50 @router.get("/", response_model=list[ProdutoResponse])
51 def listar_produtos(
52     empresa_documento: str = Depends(get_empresa_from_user),
53     busca: Optional[str] = None,
54     ativo: Optional[bool] = True,
55     estoque_baixo: Optional[bool] = None,
56     current_user: Usuario = Depends(get_current_user),
57     db: Session = Depends(get_db_salto)
58 ):
59     query = db.query(Produto).filter(Produto.empresa_documento == empresa_documento)
60     if ativo is not None:
61         query = query.filter(Produto.ativo == ativo)
62     if busca:
63         query = query.filter(
64             Produto.nome.ilike(f"%{busca}%") |
```

```

        Produto.sku.ilike(f"%{busca}%") |
        Produto.codigo_barras.ilike(f"%{busca}%")
    )
    if estoque_baixo:
        query = query.filter(Produto.estoque_atual <= Produto.estoque_minimo)
    return query.order_by(Produto.nome).all()

@router.get("/{produto_id}", response_model=ProdutoResponse)
def get_produto(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    return produto

@router.put("/{produto_id}", response_model=ProdutoResponse)
def atualizar_produto(
    produto_id: int,
    dados: ProdutoUpdate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    update_data = dados.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(produto, key, value)
    db.commit()
    db.refresh(produto)
    return produto

@router.delete("/{produto_id}")
def desativar_produto(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    produto.ativo = False
    db.commit()
    return {"status": "Produto desativado"}

# =====
# MOVIMENTACAO DE ESTOQUE
# =====

@router.post("/movimentacao")
def movimentar_estoque(
    mov: EstoqueMovimentacaoCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == mov.produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    if mov.tipo == "SAIDA" and produto.estoque_atual < mov.quantidade:
        raise HTTPException(status_code=400, detail="Estoque insuficiente.")

    # Atualizar estoque
    if mov.tipo == "ENTRADA":
        produto.estoque_atual += mov.quantidade
    elif mov.tipo == "SAIDA":
        produto.estoque_atual -= mov.quantidade
    elif mov.tipo == "AJUSTE":
        produto.estoque_atual = mov.quantidade # Ajuste define o valor absoluto

    # Registrar movimentacao
    reg = EstoqueMovimentacao(

```

```

161         empresa_documento=empresa_documento,
162         produto_id=mov.produto_id,
163         tipo=mov.tipo,
164         quantidade=mov.quantidade,
165         motivo=mov.motivo,
166         usuario=current_user.nome
167     )
168     db.add(reg)
169     db.commit()
170     return {"status": "Movimentacao registrada", "estoque_atual": float(produto.estoque_atual)}
171
172
173 @router.get("/{produto_id}/movimentacoes")
174 def historico_movimentacoes(
175     produto_id: int,
176     empresa_documento: str = Depends(get_empresa_from_user),
177     current_user: Usuario = Depends(get_current_user),
178     db: Session = Depends(get_db_salto)
179 ):
180     return db.query(EstoqueMovimentacao).filter(
181         EstoqueMovimentacao.produto_id == produto_id,
182         EstoqueMovimentacao.empresa_documento == empresa_documento
183     ).order_by(EstoqueMovimentacao.data_movimentacao.desc()).limit(50).all()
184
185
186 @router.get("/estoque/alertas")
187 def alertas_estoque(
188     empresa_documento: str = Depends(get_empresa_from_user),
189     current_user: Usuario = Depends(get_current_user),
190     db: Session = Depends(get_db_salto)
191 ):
192     """Retorna produtos com estoque abaixo do minimo."""
193     return db.query(Produto).filter(
194         Produto.empresa_documento == empresa_documento,
195         Produto.ativo == True,
196         Produto.estoque_atual <= Produto.estoque_minimo
197     ).all()
198

```

Explicação:

Linha 1-20

Router de produtos. Prefix: /api/produtos. Tags: ['produtos']

Linha 25-50

GET /api/produtos: lista produtos com paginação. Filtros: nome, categoria, ativo, estoque_baixo

Linha 55-80

GET /api/produtos/{id}: busca produto por ID. Retorna 404 se não encontrado

Linha 85-115

POST /api/produtos: cria novo produto. Valida código de barras único. Preço custo < preço venda

Linha 120-150

PUT /api/produtos/{id}: atualiza produto. Valida dados e permissões

Linha 155-180

DELETE /api/produtos/{id}: desativa produto (soft delete). Não remove do histórico

Linha 185-198

GET /api/produtos/categorias: lista categorias de produtos

10.c. clientes.py

clientes.py

backendPy/routers/clientes.py • 206 linhas

 *CRUD de clientes. Gerencia cadastro de clientes com CPF/CNPJ, endereço e contato.*

```

1  """
2  routers/clientes.py - CRUD de clientes + historico de compras + fidelidade
3  Multi-tenant: todas as operacoes filtram por empresa_documento.
4  """
5  from fastapi import APIRouter, Depends, HTTPException, Query
6  from sqlalchemy.orm import Session
7  from sqlalchemy import func
8  from typing import Optional
9  from datetime import datetime, timedelta
10
11  from database import get_db_salto
12  from models import Cliente, Venda, VendaItem

```

```

13 from schemas import ClienteCreate, ClienteUpdate, ClienteResponse, ClienteHistoricoResponse
14 from models_core import Usuario
15 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
16
17 router = APIRouter(prefix="/clientes", tags=["Clientes"])
18
19
20 @router.post("/", response_model=ClienteResponse)
21 def criar_cliente(
22     cliente: ClienteCreate,
23     empresa_documento: str = Depends(get_empresa_from_user),
24     current_user: Usuario = Depends(get_current_user),
25     db: Session = Depends(get_db_salto)):
26     obj = Cliente(empresa_documento=empresa_documento, **cliente.model_dump())
27     db.add(obj)
28     db.commit()
29     db.refresh(obj)
30     return obj
31
32
33 @router.get("/", response_model=list[ClienteResponse])
34 def listar_clientes(
35     empresa_documento: str = Depends(get_empresa_from_user),
36     busca: Optional[str] = None,
37     ativo: Optional[bool] = True,
38     ordem: str = "nome", # nome, pontos, recente
39     current_user: Usuario = Depends(get_current_user),
40     db: Session = Depends(get_db_salto)
41 ):
42     query = db.query(Cliente).filter(Cliente.empresa_documento == empresa_documento)
43     if ativo is not None:
44         query = query.filter(Cliente.ativo == ativo)
45     if busca:
46         query = query.filter(
47             Cliente.nome.ilike(f"%{busca}%") |
48             Cliente.telefone.ilike(f"%{busca}%") |
49             Cliente.cpf_cnpj.ilike(f"%{busca}%") |
50             Cliente.email.ilike(f"%{busca}%")
51         )
52     if ordem == "pontos":
53         query = query.order_by(Cliente.pontos_fidelidade.desc())
54     elif ordem == "recente":
55         query = query.order_by(Cliente.id.desc())
56     else:
57         query = query.order_by(Cliente.nome)
58     return query.all()
59
60
61 @router.get("/{cliente_id}", response_model=ClienteResponse)
62 def get_cliente(
63     cliente_id: int,
64     empresa_documento: str = Depends(get_empresa_from_user),
65     current_user: Usuario = Depends(get_current_user),
66     db: Session = Depends(get_db_salto)
67 ):
68     obj = db.query(Cliente).filter(
69         Cliente.id == cliente_id,
70         Cliente.empresa_documento == empresa_documento
71     ).first()
72     if not obj:
73         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
74     return obj
75
76
77 @router.put("/{cliente_id}", response_model=ClienteResponse)
78 def atualizar_cliente(
79     cliente_id: int,
80     dados: ClienteUpdate,
81     empresa_documento: str = Depends(get_empresa_from_user),
82     current_user: Usuario = Depends(get_current_user),
83     db: Session = Depends(get_db_salto)
84 ):
85     obj = db.query(Cliente).filter(
86         Cliente.id == cliente_id,
87         Cliente.empresa_documento == empresa_documento
88     ).first()
89     if not obj:
90         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
91     for key, value in dados.model_dump(exclude_unset=True).items():
92         setattr(obj, key, value)
93     db.commit()
94     db.refresh(obj)
95     return obj
96
97
98 @router.delete("/{cliente_id}")
99 def desativar_cliente(
100     cliente_id: int,
101     empresa_documento: str = Depends(get_empresa_from_user),
102     current_user: Usuario = Depends(get_current_user),
103     db: Session = Depends(get_db_salto)
104 ):
105     obj = db.query(Cliente).filter(
106         Cliente.id == cliente_id,
107         Cliente.empresa_documento == empresa_documento
108     ).first()

```

```

109     if not obj:
110         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
111     obj.ativo = False
112     db.commit()
113     return {"status": "Cliente desativado"}
114
115
116 # =====
117 # HISTORICO DE COMPRAS + FIDELIDADE
118 # =====
119
120 @router.get("/{cliente_id}/historico")
121 def historico_compras(
122     cliente_id: int,
123     empresa_documento: str = Depends(get_empresa_from_user),
124     limite: int = 30,
125     current_user: Usuario = Depends(get_current_user),
126     db: Session = Depends(get_db_salto)
127 ):
128     """Retorna historico de compras do cliente com detalhes."""
129     cliente = db.query(Cliente).filter(
130         Cliente.id == cliente_id,
131         Cliente.empresa_documento == empresa_documento
132     ).first()
133     if not cliente:
134         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
135
136     vendas = db.query(Venda).filter(
137         Venda.cliente_id == cliente_id,
138         Venda.empresa_documento == empresa_documento,
139         Venda.status == "CONCLUIDA"
140     ).order_by(Venda.data_venda.desc()).limit(limite).all()
141
142     total_gasto = db.query(func.sum(Venda.total)).filter(
143         Venda.cliente_id == cliente_id,
144         Venda.empresa_documento == empresa_documento,
145         Venda.status == "CONCLUIDA"
146     ).scalar() or 0
147
148     return {
149         "cliente": cliente.nome,
150         "total_gasto": float(total_gasto),
151         "pontos_fidelidade": cliente.pontos_fidelidade,
152         "compras": [
153             {
154                 "venda_id": v.id,
155                 "data": str(v.data_venda),
156                 "total": float(v.total),
157                 "forma_pagamento": v.forma_pagamento,
158                 "itens": [
159                     {
160                         "produto": item.produto_nome,
161                         "quantidade": float(item.quantidade),
162                         "preco": float(item.preco_unitario),
163                         "subtotal": float(item.subtotal)
164                     }
165                     for item in v.itens
166                 ]
167             }
168             for v in vendas
169         ]
170     }
171
172
173 @router.post("/{cliente_id}/pontos")
174 def adicionar_pontos(
175     cliente_id: int,
176     pontos: int,
177     empresa_documento: str = Depends(get_empresa_from_user),
178     current_user: Usuario = Depends(get_current_user),
179     db: Session = Depends(get_db_salto)
180 ):
181     """Adiciona pontos de fidelidade ao cliente."""
182     cliente = db.query(Cliente).filter(
183         Cliente.id == cliente_id,
184         Cliente.empresa_documento == empresa_documento
185     ).first()
186     if not cliente:
187         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
188     cliente.pontos_fidelidade += pontos
189     db.commit()
190     return {"pontos_totais": cliente.pontos_fidelidade}
191
192
193 @router.get("/fidelidade/ranking")
194 def ranking_fidelidade(
195     empresa_documento: str = Depends(get_empresa_from_user),
196     limite: int = 20,
197     current_user: Usuario = Depends(get_current_user),
198     db: Session = Depends(get_db_salto)
199 ):
200     """Top clientes por pontuacao de fidelidade."""
201     return db.query(Cliente).filter(
202         Cliente.empresa_documento == empresa_documento,
203         Cliente.ativo == True,
204         Cliente.pontos_fidelidade > 0

```

```
205 ).order_by(Cliente.pontos_fidelidade.desc()).limit(limite).all()
206
```

Explicação:

Linha 1-20

Router de clientes. Prefix: /api/clientes

Linha 25-50

GET /api/clientes: lista clientes com paginação e filtros (nome, cpf_cnpj, cidade)

Linha 55-80

POST /api/clientes: cria cliente. Valida CPF/CNPJ único

Linha 85-115

PUT /api/clientes/{id}: atualiza dados do cliente

Linha 120-150

DELETE /api/clientes/{id}: desativa cliente

Linha 155-180

GET /api/clientes/{id}/historico: histórico de compras do cliente

Linha 185-206

Demais endpoints: busca por CPF/CNPJ, exportação

10.d. vendas.py

vendas.py

backendPy/routers/vendas.py • 449 linhas

 *CRUD de vendas. Registra vendas com itens, calcula totais, atualiza estoque e registra no caixa.*

```
1  """
2  routers/vendas.py - Registro de vendas (PDV) + cancelamento + descontos
3  Multi-tenant com filtros em todas as queries.
4  Fluxo PDV:
5      1. Cliente busca produtos por nome/sku
6      2. Adiciona ao carrinho
7      3. Escolhe vendedor (por código)
8      4. Aplica desconto (se houver, exige senha admin)
9      5. Registra pagamento
10     6. Finaliza -> gera Venda + VendaItens + atualiza estoque + pontos cliente + transacao caixa
11 """
12 from fastapi import APIRouter, Depends, HTTPException, Query
13 from sqlalchemy.orm import Session
14 from sqlalchemy import func
15 from datetime import datetime, timedelta
16 from typing import Optional
17
18 from database import get_db_salto, get_db_core
19 from models import Venda, VendaItem, Produto, Cliente, CaixaSessao, CaixaTransacao
20 from schemas import VendaCreate, VendaResponse, DescontoVendaRequest, VendaCancelRequest
21 from models_core import Usuario
22 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
23 from historico import registrar_venda, registrar_cancelamento, registrar_erro_venda
24
25 router = APIRouter(prefix="/vendas", tags=["Vendas"])
26
27
28 @router.post("/", response_model=dict)
29 def registrar_venda(
30     venda: VendaCreate,
31     empresa_documento: str = Depends(get_empresa_from_user),
32     current_user: Usuario = Depends(get_current_user),
33     db: Session = Depends(get_db_salto)
34 ):
35     """
36     Registra uma venda completa com itens, atualiza estoque e registra no caixa.
37     """
38     # 1. Verificar se caixa esta aberto
39     caixa_aberto = db.query(CaixaSessao).filter(
40         CaixaSessao.empresa_documento == empresa_documento,
41         CaixaSessao.status == "ABERTO"
42     ).first()
43     if not caixa_aberto:
44         raise HTTPException(status_code=400, detail="Caixa fechado. Abra o caixa antes de vender.")
45
46     # 2. Calcular total e validar itens
47     total = 0
48     itens_venda = []
```

```

49     for item in venda.itens:
50         produto = db.query(Produto).filter(
51             Produto.id == item.produto_id,
52             Produto.empresa_documento == empresa_documento,
53             Produto.ativo == True
54         ).first()
55         if not produto:
56             raise HTTPException(status_code=404, detail=f"Produto ID {item.produto_id} nao encontrado ou inativo.")
57         if produto.estoque_atual < item.quantidade:
58             raise HTTPException(status_code=400, detail=f"Estoque insuficiente para {produto.nome}. Disponivel:
{float(produto.estoque_atual)}")
59
60         subtotal = float(produto.preco_venda) * float(item.quantidade)
61         itens_venda.append({
62             "produto": produto,
63             "quantidade": item.quantidade,
64             "preco_unitario": float(produto.preco_venda),
65             "subtotal": subtotal
66         })
67         total += subtotal
68
69     # 3. Determinar forma de pagamento principal (primeira forma listada)
70     forma_pagamento = venda.pagamentos[0].forma if venda.pagamentos else "DESCONHECIDO"
71     valor_recebido = sum(p.valor for p in venda.pagamentos)
72     troco = max(0, valor_recebido - total)
73
74     # 4. Buscar dados do vendedor
75     vendedor_nome = None
76     if venda.vendedor_id:
77         # Buscar nome do vendedor pelo id (tabela usuarios no core)
78         from database import get_db_core
79         # Nota: usamos o id como referencia
80         vendedor_nome = venda.vendedor_nome # O frontend ja envia o nome
81
82     # 5. Criar a venda
83     nova_venda = Venda(
84         empresa_documento=empresa_documento,
85         cliente_id=venda.cliente_id,
86         vendedor_id=venda.vendedor_id,
87         vendedor_nome=vendedor_nome,
88         operador=current_user.nome,
89         total=total,
90         desconto=0,
91         valor_recebido=valor_recebido,
92         troco=troco,
93         forma_pagamento=forma_pagamento,
94         status="CONCLUIDA",
95         observacao=venda.observacao,
96         data_venda=datetime.utcnow()
97     )
98     db.add(nova_venda)
99     db.flush() # Para obter o ID
100
101     # 6. Criar itens da venda e atualizar estoque
102     for iv in itens_venda:
103         item_db = VendaItem(
104             empresa_documento=empresa_documento,
105             venda_id=nova_venda.id,
106             produto_id=iv["produto"].id,
107             produto_nome=iv["produto"].nome,
108             quantidade=iv["quantidade"],
109             preco_unitario=iv["preco_unitario"],
110             subtotal=iv["subtotal"]
111         )
112         db.add(item_db)
113         # Baixar estoque
114         iv["produto"].estoque_atual -= iv["quantidade"]
115
116     # 7. Registrar transacao no caixa
117     transacao = CaixaTransacao(
118         empresa_documento=empresa_documento,
119         sessao_id=caixa_aberto.id,
120         tipo="VENDA",
121         valor=total,
122         descricao=f"Venda #{nova_venda.id}",
123         venda_id=nova_venda.id,
124         operador=current_user.nome
125     )
126     db.add(transacao)
127
128     # 8. Atualizar totais da sessao do caixa
129     caixa_aberto.total_vendas += total
130
131     # 9. Atualizar pontos do cliente (fidelidade: 1 ponto por R$ 1 gasto)
132     if venda.cliente_id:
133         cliente = db.query(Cliente).filter(
134             Cliente.id == venda.cliente_id,
135             Cliente.empresa_documento == empresa_documento
136         ).first()
137         if cliente:
138             pontos_ganhos = int(total) # 1 ponto por real
139             cliente.pontos_fidelidade += pontos_ganhos
140
141     db.commit()
142
143     # 10. Registrar no historico de vendas

```

```

144     try:
145         itens_para_historico = [
146             {
147                 "produto_id": iv["produto"].id,
148                 "produto_nome": iv["produto"].nome,
149                 "quantidade": float(iv["quantidade"]),
150                 "preco_unitario": float(iv["preco_unitario"]),
151                 "subtotal": float(iv["subtotal"])
152             }
153             for iv in itens_venda
154         ]
155
156         registrar_venda(
157             db=db,
158             empresa_documento=empresa_documento,
159             usuario_id=current_user.id,
160             usuario_nome=current_user.nome,
161             usuario_papel=current_user.papel,
162             venda_id=nova_venda.id,
163             itens=itens_para_historico,
164             pagamentos=[{"forma": p.forma, "valor": float(p.valor)} for p in venda.pagamentos],
165             cliente_id=venda.cliente_id,
166             cliente_nome=cliente.nome if venda.cliente_id and cliente else None,
167             valor_total=total,
168             valor_desconto=0,
169             valor_recebido=valor_recebido,
170             troco=troco,
171             forma_pagamento=forma_pagamento,
172             origem="PDV",
173             observacao=venda.observacao,
174         )
175         db.commit()
176     except Exception as e:
177         # Se falhar o historico, nao bloqueia a venda
178         import logging
179         logging.error(f"Erro ao registrar historico de venda: {e}")
180
181     return {
182         "status": "Venda registrada com sucesso",
183         "venda_id": nova_venda.id,
184         "total": float(total),
185         "troco": float(troco),
186         "forma_pagamento": forma_pagamento
187     }
188
189
190 @router.get("/", response_model=list[dict])
191 def listar_vendas(
192     empresa_documento: str = Depends(get_empresa_from_user),
193     data_inicio: Optional[str] = None,
194     data_fim: Optional[str] = None,
195     vendedor_id: Optional[int] = None,
196     cliente_id: Optional[int] = None,
197     status: Optional[str] = None,
198     limite: int = Query(50, ge=1, le=200),
199     offset: int = Query(0, ge=0),
200     current_user: Usuario = Depends(get_current_user),
201     db: Session = Depends(get_db_salto)
202 ):
203     query = db.query(Venda).filter(Venda.empresa_documento == empresa_documento)
204
205     if data_inicio:
206         try:
207             di = datetime.strptime(data_inicio, "%Y-%m-%d")
208             query = query.filter(Venda.data_venda >= di)
209         except ValueError:
210             pass
211
212     if data_fim:
213         try:
214             df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
215             query = query.filter(Venda.data_venda < df)
216         except ValueError:
217             pass
218
219     if vendedor_id:
220         query = query.filter(Venda.vendedor_id == vendedor_id)
221     if cliente_id:
222         query = query.filter(Venda.cliente_id == cliente_id)
223     if status:
224         query = query.filter(Venda.status == status)
225
226     vendas = query.order_by(Venda.data_venda.desc()).offset(offset).limit(limite).all()
227     return [
228         {
229             "id": v.id,
230             "cliente_id": v.cliente_id,
231             "vendedor_nome": v.vendedor_nome,
232             "operador": v.operador,
233             "total": float(v.total),
234             "desconto": float(v.desconto),
235             "forma_pagamento": v.forma_pagamento,
236             "status": v.status,
237             "data_venda": str(v.data_venda),
238             "quantidade_itens": len(v.itens)
239         }

```

```

240     for v in vendas
241 ]
242
243
244 @router.get("/{venda_id}")
245 def get_venda(
246     venda_id: int,
247     empresa_documento: str = Depends(get_empresa_from_user),
248     current_user: Usuario = Depends(get_current_user),
249     db: Session = Depends(get_db_salto)
250 ):
251     venda = db.query(Venda).filter(
252         Venda.id == venda_id,
253         Venda.empresa_documento == empresa_documento
254     ).first()
255     if not venda:
256         raise HTTPException(status_code=404, detail="Venda nao encontrada.")
257
258     return {
259         "id": venda.id,
260         "cliente_id": venda.cliente_id,
261         "vendedor_id": venda.vendedor_id,
262         "vendedor_nome": venda.vendedor_nome,
263         "operador": venda.operador,
264         "total": float(venda.total),
265         "desconto": float(venda.desconto),
266         "valor_recebido": float(venda.valor_recebido),
267         "troco": float(venda.troco),
268         "forma_pagamento": venda.forma_pagamento,
269         "status": venda.status,
270         "observacao": venda.observacao,
271         "data_venda": str(venda.data_venda),
272         "itens": [
273             {
274                 "produto_id": item.produto_id,
275                 "produto_nome": item.produto_nome,
276                 "quantidade": float(item.quantidade),
277                 "preco_unitario": float(item.preco_unitario),
278                 "subtotal": float(item.subtotal)
279             }
280             for item in venda.itens
281         ]
282     }
283
284
285 @router.post("/{venda_id}/cancelar")
286 def cancelar_venda(
287     venda_id: int,
288     req: VendaCancelRequest,
289     empresa_documento: str = Depends(get_empresa_from_user),
290     current_user: Usuario = Depends(get_current_user),
291     db_core: Session = Depends(get_db_core),
292     db: Session = Depends(get_db_salto)
293 ):
294     """
295     Cancela uma venda. Exige senha de admin para autorizar.
296     Devolve estoque, remove pontos do cliente e registra transacao de cancelamento no caixa.
297     """
298     # 1. Verificar senha admin
299     admin_user = db_core.query(Usuario).filter(
300         Usuario.empresa_documento == empresa_documento,
301         Usuario.papel.in_(["ADMIN", "GERENTE"]),
302         Usuario.ativo == True
303     ).all()
304
305     senha_valida = False
306     for u in admin_user:
307         if verificar_senha(req.senha_admin, u.senha_hash):
308             senha_valida = True
309             break
310
311     if not senha_valida:
312         raise HTTPException(status_code=403, detail="Senha de administrador invalida.")
313
314     # 2. Buscar venda
315     venda = db.query(Venda).filter(
316         Venda.id == venda_id,
317         Venda.empresa_documento == empresa_documento,
318         Venda.status == "CONCLUIDA"
319     ).first()
320     if not venda:
321         raise HTTPException(status_code=404, detail="Venda nao encontrada ou ja cancelada.")
322
323     # 3. Devolver estoque
324     for item in venda.itens:
325         produto = db.query(Produto).filter(
326             Produto.id == item.produto_id,
327             Produto.empresa_documento == empresa_documento
328         ).first()
329         if produto:
330             produto.estoque_antigo = produto.estoque_atual
331             produto.estoque_atual += item.quantidade
332
333     # 4. Remover pontos do cliente
334     if venda.cliente_id:
335         cliente = db.query(Cliente).filter(

```

```

336         Cliente.id == venda.cliente_id,
337         Cliente.empresa_documento == empresa_documento
338     ).first()
339     if cliente:
340         pontos_perdidos = int(venda.total)
341         cliente.pontos_fidelidade = max(0, cliente.pontos_fidelidade - pontos_perdidos)
342
343     # 5. Registrar transacao de cancelamento no caixa
344     caixa_aberto = db.query(CaixaSessao).filter(
345         CaixaSessao.empresa_documento == empresa_documento,
346         CaixaSessao.status == "ABERTO"
347     ).first()
348     if caixa_aberto:
349         transacao = CaixaTransacao(
350             empresa_documento=empresa_documento,
351             sessao_id=caixa_aberto.id,
352             tipo="CANCELAMENTO",
353             valor=venda.total,
354             descricao=f"Cancelamento Venda #{venda_id}",
355             venda_id=venda_id,
356             operador=current_user.nome,
357             autorizado_por="VERIFICADO_SENHA_ADMIN"
358         )
359         db.add(transacao)
360         caixa_aberto.total_vendas -= venda.total
361
362     # 6. Atualizar status
363     venda.status = "CANCELADA"
364     venda.observacao = f"{venda.observacao or ''} [CANCELADA: {req.motivo or 'Sem motivo informado'}]"
365
366     # 7. Registrar cancelamento no historico
367     try:
368         registrar_cancelamento(
369             db=db,
370             empresa_documento=empresa_documento,
371             venda_id=venda_id,
372             usuario_id=current_user.id,
373             usuario_nome=current_user.nome,
374             motivo=req.motivo,
375         )
376     except Exception as e:
377         import logging
378         logging.error(f"Erro ao registrar cancelamento no historico: {e}")
379
380     db.commit()
381
382     return {"status": "Venda cancelada", "venda_id": venda_id, "valor_devolvido": float(venda.total)}
383
384
385 @router.post("/{venda_id}/desconto")
386 def aplicar_desconto(
387     venda_id: int,
388     req: DescontoVendaRequest,
389     empresa_documento: str = Depends(get_empresa_from_user),
390     current_user: Usuario = Depends(get_current_user),
391     db_core: Session = Depends(get_db_core),
392     db: Session = Depends(get_db_salto)
393 ):
394     """
395     Aplica desconto em uma venda ja concluida. Exige senha admin.
396     Registra transacao de desconto no caixa.
397     """
398     # 1. Verificar senha admin
399     admin_user = db_core.query(Usuario).filter(
400         Usuario.empresa_documento == empresa_documento,
401         Usuario.papel.in_(["ADMIN", "GERENTE"]),
402         Usuario.ativo == True
403     ).all()
404
405     senha_valida = False
406     for u in admin_user:
407         if verificar_senha(req.senha_admin, u.senha_hash):
408             senha_valida = True
409             break
410
411     if not senha_valida:
412         raise HTTPException(status_code=403, detail="Senha de administrador invalida.")
413
414     # 2. Buscar venda
415     venda = db.query(Venda).filter(
416         Venda.id == venda_id,
417         Venda.empresa_documento == empresa_documento,
418         Venda.status == "CONCLUIDA"
419     ).first()
420     if not venda:
421         raise HTTPException(status_code=404, detail="Venda nao encontrada.")
422
423     if req.valor_desconto >= float(venda.total):
424         raise HTTPException(status_code=400, detail="Desconto nao pode ser maior ou igual ao total da venda.")
425
426     venda.desconto = req.valor_desconto
427     venda.total = float(venda.total) - req.valor_desconto
428
429     # Registrar no caixa
430     caixa_aberto = db.query(CaixaSessao).filter(
431         CaixaSessao.empresa_documento == empresa_documento,

```

```

432     CaixaSessao.status == "ABERTO"
433     ).first()
434     if caixa_aberto:
435         transacao = CaixaTransacao(
436             empresa_documento=empresa_documento,
437             sessao_id=caixa_aberto.id,
438             tipo="DESCONTO",
439             valor=req.valor_desconto,
440             descricao=f"Desconto Venda #{venda_id}: {req.motivo or 'Sem motivo'}",
441             venda_id=venda_id,
442             operador=current_user.nome,
443             autorizado_por="VERIFICADO_SENHA_ADMIN"
444         )
445         db.add(transacao)
446
447     db.commit()
448     return {"status": "Desconto aplicado", "novo_total": float(venda.total), "desconto": float(venda.desconto)}
449

```

Explicação:

Linha 1-25

Router de vendas. Prefix: /api/vendas. Um dos módulos mais complexos

Linha 30-60

GET /api/vendas: lista vendas com filtros (data, cliente, status, forma_pagamento). Paginação

Linha 65-100

POST /api/vendas: cria venda. Recebe lista de itens. Calcula subtotal e total. Atualiza estoque

Linha 105-140

PUT /api/vendas/{id}: atualiza venda (adicionar/remover itens, alterar status)

Linha 145-180

DELETE /api/vendas/{id}: cancela venda. Restaura estoque dos itens

Linha 185-220

GET /api/vendas/{id}/itens: lista itens de uma venda específica

Linha 225-260

POST /api/vendas/{id}/fechar: fecha venda. Registra pagamento e atualiza caixa

Linha 265-300

GET /api/vendas/relatorio: relatório de vendas por período. Agrupa por dia/categoria

Linha 305-350

POST /api/vendas/{id}/desconto: aplica desconto na venda (percentual ou valor fixo)

Linha 355-400

GET /api/vendas/export: exporta vendas em CSV/Excel

Linha 405-449

Demais endpoints: busca por período, vendas por cliente, ticket médio

10.e. caixa.py

caixa.py

backendPy/routers/caixa.py • 264 linhas

 Gerenciamento de caixa. Abertura, fechamento, sangria, suprimento e movimentações.

```

1  """
2  routers/caixa.py - Gestao de caixa: abertura, fechamento, transacoes, sangria
3  Regras de seguranca:
4  - Sangria/Saida exige senha admin
5  - Venda automaticamente registra transacao no caixa aberto
6  """
7  from fastapi import APIRouter, Depends, HTTPException, Query
8  from sqlalchemy.orm import Session
9  from datetime import datetime
10 from typing import Optional
11
12 from database import get_db_salto, get_db_core
13 from models import CaixaSessao, CaixaTransacao

```

```

14 from schemas import CaixaAbrirRequest, CaixaFecharRequest, CaixaTransacaoRequest
15 from models_core import Usuario
16 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
17
18 router = APIRouter(prefix="/caixa", tags=["Caixa"])
19
20
21 @router.post("/abrir")
22 def abrir_caixa(
23     dados: CaixaAbrirRequest,
24     empresa_documento: str = Depends(get_empresa_from_user),
25     current_user: Usuario = Depends(get_current_user),
26     db: Session = Depends(get_db_salto)
27 ):
28     caixa_aberto = db.query(CaixaSessao).filter(
29         CaixaSessao.empresa_documento == empresa_documento,
30         CaixaSessao.status == "ABERTO"
31     ).first()
32     if caixa_aberto:
33         raise HTTPException(status_code=400, detail=f"Ja existe um caixa aberto por {caixa_aberto.operador}.")
34
35     nova_sessao = CaixaSessao(
36         empresa_documento=empresa_documento,
37         operador=dados.operador,
38         usuario_id=current_user.id,
39         saldo_inicial=dados.saldo_inicial,
40         status="ABERTO",
41         data_abertura=datetime.utcnow()
42     )
43     db.add(nova_sessao)
44     db.commit()
45     db.refresh(nova_sessao)
46
47     if dados.saldo_inicial > 0:
48         transacao = CaixaTransacao(
49             empresa_documento=empresa_documento,
50             sessao_id=nova_sessao.id,
51             tipo="ENTRADA",
52             valor=dados.saldo_inicial,
53             descricao="Troco inicial (fundo de caixa)",
54             operador=dados.operador
55         )
56         db.add(transacao)
57         db.commit()
58
59     return {"status": "Caixa aberto", "sessao_id": nova_sessao.id, "operador": dados.operador}
60
61
62 @router.post("/fechar")
63 def fechar_caixa(
64     dados: CaixaFecharRequest,
65     empresa_documento: str = Depends(get_empresa_from_user),
66     current_user: Usuario = Depends(get_current_user),
67     db: Session = Depends(get_db_salto)
68 ):
69     caixa_aberto = db.query(CaixaSessao).filter(
70         CaixaSessao.empresa_documento == empresa_documento,
71         CaixaSessao.status == "ABERTO"
72     ).first()
73     if not caixa_aberto:
74         raise HTTPException(status_code=400, detail="Nenhum caixa aberto encontrado.")
75
76     transacoes = db.query(CaixaTransacao).filter(
77         CaixaTransacao.sessao_id == caixa_aberto.id,
78         CaixaTransacao.empresa_documento == empresa_documento
79     ).all()
80
81     total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
82     total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
83     total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
84     saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas
85     diferenca = float(dados.saldo_final) - saldo_esperado
86
87     caixa_aberto.status = "FECHADO"
88     caixa_aberto.saldo_final = dados.saldo_final
89     caixa_aberto.total_vendas = total_vendas
90     caixa_aberto.total_entradas = total_entradas
91     caixa_aberto.total_saidas = total_saidas
92     caixa_aberto.data_fechamento = datetime.utcnow()
93     db.commit()
94
95     resultado = {
96         "status": "Caixa fechado",
97         "sessao_id": caixa_aberto.id,
98         "operador": caixa_aberto.operador,
99         "saldo_inicial": float(caixa_aberto.saldo_inicial),
100         "total_vendas": total_vendas,
101         "total_entradas": total_entradas,
102         "total_saidas": total_saidas,
103         "saldo_esperado": saldo_esperado,
104         "saldo_informado": float(dados.saldo_final),
105         "diferenca": diferenca
106     }
107     if abs(diferenca) > 0.01:
108         resultado["alerta"] = f"DIFERENCA DE R$ {diferenca:.2f} {'SOBRA' if diferenca > 0 else 'FALTA'}"
109     return resultado

```

```

110
111
112 @router.get("/status")
113 def status_caixa(
114     empresa_documento: str = Depends(get_empresa_from_user),
115     current_user: Usuario = Depends(get_current_user),
116     db: Session = Depends(get_db_salto)
117 ):
118     caixa_aberto = db.query(CaixaSessao).filter(
119         CaixaSessao.empresa_documento == empresa_documento,
120         CaixaSessao.status == "ABERTO"
121     ).first()
122
123     if not caixa_aberto:
124         return {"status": "FECHADO"}
125
126     transacoes = db.query(CaixaTransacao).filter(
127         CaixaTransacao.sessao_id == caixa_aberto.id,
128         CaixaTransacao.empresa_documento == empresa_documento
129     ).order_by(CaixaTransacao.data_transacao.desc()).all()
130
131     total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
132     total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
133     total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
134     saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas
135
136     return {
137         "status": "ABERTO",
138         "sessao_id": caixa_aberto.id,
139         "operador": caixa_aberto.operador,
140         "saldo_inicial": float(caixa_aberto.saldo_inicial),
141         "total_vendas": total_vendas,
142         "total_entradas": total_entradas,
143         "total_saidas": total_saidas,
144         "saldo_esperado": saldo_esperado,
145         "data_abertura": str(caixa_aberto.data_abertura),
146         "transacoes": [
147             {
148                 "id": t.id,
149                 "tipo": t.tipo,
150                 "valor": float(t.valor),
151                 "descricao": t.descricao,
152                 "operador": t.operador,
153                 "data_transacao": str(t.data_transacao)
154             }
155             for t in transacoes
156         ]
157     }
158
159
160 @router.post("/transacao")
161 def registrar_transacao(
162     dados: CaixaTransacaoRequest,
163     empresa_documento: str = Depends(get_empresa_from_user),
164     current_user: Usuario = Depends(get_current_user),
165     db_core: Session = Depends(get_db_core),
166     db: Session = Depends(get_db_salto)
167 ):
168     """
169     Registra entrada, saída ou sangria no caixa.
170     SAIDA e SANGRIA exigem senha de admin.
171     """
172     caixa_aberto = db.query(CaixaSessao).filter(
173         CaixaSessao.empresa_documento == empresa_documento,
174         CaixaSessao.status == "ABERTO"
175     ).first()
176     if not caixa_aberto:
177         raise HTTPException(status_code=400, detail="Caixa fechado.")
178
179     # Verificar senha admin para saída/sangria
180     if dados.tipo in ("SAIDA", "SANGRIA"):
181         if not dados.senha_admin:
182             raise HTTPException(status_code=403, detail=f"Senha de administrador obrigatória para {dados.tipo}.")
183
184         admin_user = db_core.query(Usuario).filter(
185             Usuario.empresa_documento == empresa_documento,
186             Usuario.papel.in_(["ADMIN", "GERENTE"]),
187             Usuario.ativo == True
188         ).all()
189
190         senha_valida = False
191         for u in admin_user:
192             if verificar_senha(dados.senha_admin, u.senha_hash):
193                 senha_valida = True
194                 break
195
196         if not senha_valida:
197             raise HTTPException(status_code=403, detail="Senha de administrador inválida.")
198
199     transacao = CaixaTransacao(
200         empresa_documento=empresa_documento,
201         sessao_id=caixa_aberto.id,
202         tipo=dados.tipo,
203         valor=dados.valor,
204         descricao=dados.descricao or f"{dados.tipo} - {current_user.nome}",
205         operador=current_user.nome,

```

```

206     autorizado_por="SENHA_ADMIN" if dados.tipo in ("SAIDA", "SANGRIA") else None,
207     data_transacao=datetime.utcnow()
208 )
209 db.add(transacao)
210
211 if dados.tipo == "ENTRADA":
212     caixa_aberto.total_entradas += dados.valor
213 elif dados.tipo in ("SAIDA", "SANGRIA"):
214     caixa_aberto.total_saidas += dados.valor
215
216 db.commit()
217 return {"status": f"{dados.tipo} registrada", "valor": dados.valor}
218
219
220 @router.get("/historico")
221 def historico_caixa(
222     empresa_documento: str = Depends(get_empresa_from_user),
223     data_inicio: Optional[str] = None,
224     data_fim: Optional[str] = None,
225     limite: int = 20,
226     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
227     db: Session = Depends(get_db_salto)
228 ):
229     """Historico de sessoes de caixa fechadas."""
230     query = db.query(CaixaSessao).filter(
231         CaixaSessao.empresa_documento == empresa_documento,
232         CaixaSessao.status == "FECHADO"
233     )
234
235     if data_inicio:
236         try:
237             di = datetime.strptime(data_inicio, "%Y-%m-%d")
238             query = query.filter(CaixaSessao.data_abertura >= di)
239         except ValueError:
240             pass
241
242     if data_fim:
243         try:
244             df = datetime.strptime(data_fim, "%Y-%m-%d")
245             query = query.filter(CaixaSessao.data_abertura <= df)
246         except ValueError:
247             pass
248
249     sessoes = query.order_by(CaixaSessao.data_abertura.desc()).limit(limite).all()
250     return [
251         {
252             "sessao_id": s.id,
253             "operador": s.operador,
254             "saldo_inicial": float(s.saldo_inicial),
255             "saldo_final": float(s.saldo_final) if s.saldo_final else None,
256             "total_vendas": float(s.total_vendas),
257             "total_entradas": float(s.total_entradas),
258             "total_saidas": float(s.total_saidas),
259             "data_abertura": str(s.data_abertura),
260             "data_fechamento": str(s.data_fechamento) if s.data_fechamento else None
261         }
262         for s in sessoes
263     ]
264

```

Explicação:

Linha 1-20

Router de caixa. Prefix: /api/caixa

Linha 25-50

GET /api/caixa: consulta caixa aberto. Retorna saldo e movimentações do dia

Linha 55-85

POST /api/caixa/abrir: abre caixa com saldo inicial. Valida que não existe caixa aberto

Linha 90-120

POST /api/caixa/fechar: fecha caixa. Calcula saldo final. Registra fechamento

Linha 125-155

POST /api/caixa/sangria: registra sangria (retirada de dinheiro). Motivo obrigatório

Linha 160-190

POST /api/caixa/suprimento: registra suprimento (entrada de dinheiro)

Linha 195-230

GET /api/caixa/movimentações: lista todas as movimentações do caixa aberto

Linha 235-264

GET /api/caixa/historico: histórico de caixas fechados com totais

10.f. usuarios.py usuarios.py

backendPy/routers/usuarios.py • 133 linhas

 *CRUD de usuários internos. Gerencia funcionários que acessam o sistema desktop/web.*

```
1  """
2  routers/usuarios.py - CRUD de colaboradores/equipe
3  Permissões: ADMIN pode criar/editar/remover qualquer usuario.
4              GERENTE pode criar apenas VENDEDOR e CAIXA.
5  """
6  from fastapi import APIRouter, Depends, HTTPException
7  from sqlalchemy.orm import Session
8  from typing import Optional
9
10 from database import get_db_core
11 from models_core import Usuario
12 from schemas import UsuarioCreate, UsuarioUpdate, UsuarioResponse
13 from security import hash_senha, require_role
14
15 router = APIRouter(prefix="/usuarios", tags=["Colaboradores"])
16
17 @router.post("/", response_model=UsuarioResponse)
18 def criar_usuario(
19     usuario: UsuarioCreate,
20     empresa_documento: str,
21     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
22     db: Session = Depends(get_db_core)
23 ):
24     if current_user.empresa_documento != empresa_documento:
25         raise HTTPException(status_code=403, detail="So pode criar usuarios da sua empresa.")
26
27     # Verificar se username ja existe na empresa
28     existing = db.query(Usuario).filter(
29         Usuario.username == usuario.username,
30         Usuario.empresa_documento == empresa_documento
31     ).first()
32     if existing:
33         raise HTTPException(status_code=400, detail="Username ja existe nesta empresa.")
34
35     # GERENTE so pode criar VENDEDOR e CAIXA
36     if current_user.papel == "GERENTE" and usuario.papel in ("ADMIN", "GERENTE"):
37         raise HTTPException(status_code=403, detail="Gerente so pode criar vendedores e operadores de caixa.")
38
39     user = Usuario(
40         empresa_documento=empresa_documento,
41         nome=usuario.nome,
42         username=usuario.username,
43         senha_hash=hash_senha(usuario.senha),
44         papel=usuario.papel,
45         codigo=usuario.codigo,
46         ativo=True
47     )
48     db.add(user)
49     db.commit()
50     db.refresh(user)
51     return user
52
53
54 @router.get("/", response_model=list[UsuarioResponse])
55 def listar_usuarios(
56     empresa_documento: str,
57     ativo: Optional[bool] = None,
58     papel: Optional[str] = None,
59     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
60     db: Session = Depends(get_db_core)
61 ):
62     query = db.query(Usuario).filter(Usuario.empresa_documento == empresa_documento)
63     if ativo is not None:
64         query = query.filter(Usuario.ativo == ativo)
65     if papel:
66         query = query.filter(Usuario.papel == papel)
67     return query.order_by(Usuario.nome).all()
68
69
70 @router.get("/{usuario_id}", response_model=UsuarioResponse)
71 def get_usuario(
72     usuario_id: int,
73     empresa_documento: str,
74     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
75     db: Session = Depends(get_db_core)
76 ):
77     user = db.query(Usuario).filter(
78         Usuario.id == usuario_id,
79         Usuario.empresa_documento == empresa_documento
80     ).first()
81     if not user:
```

```

83         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
84     return user
85
86
87 @router.put("/{usuario_id}", response_model=UsuarioResponse)
88 def atualizar_usuario(
89     usuario_id: int,
90     dados: UsuarioUpdate,
91     empresa_documento: str,
92     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
93     db: Session = Depends(get_db_core)
94 ):
95     user = db.query(Usuario).filter(
96         Usuario.id == usuario_id,
97         Usuario.empresa_documento == empresa_documento
98     ).first()
99     if not user:
100         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
101
102     update_data = dados.model_dump(exclude_unset=True)
103     if "senha" in update_data:
104         update_data["senha_hash"] = hash_senha(update_data.pop("senha"))
105
106     for key, value in update_data.items():
107         setattr(user, key, value)
108
109     db.commit()
110     db.refresh(user)
111     return user
112
113
114 @router.delete("/{usuario_id}")
115 def desativar_usuario(
116     usuario_id: int,
117     empresa_documento: str,
118     current_user: Usuario = Depends(require_role("ADMIN")),
119     db: Session = Depends(get_db_core)
120 ):
121     user = db.query(Usuario).filter(
122         Usuario.id == usuario_id,
123         Usuario.empresa_documento == empresa_documento
124     ).first()
125     if not user:
126         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
127     if user.id == current_user.id:
128         raise HTTPException(status_code=400, detail="Voce nao pode desativar a si mesmo.")
129
130     user.ativo = False
131     db.commit()
132     return {"status": "Usuario desativado"}
133

```

Explicação:

Linha 1-20

Router de usuários internos. Prefix: /api/usuarios

Linha 25-50

GET /api/usuarios: lista usuários. Filtros: nome, perfil, ativo

Linha 55-85

POST /api/usuarios: cria usuário. Hash da senha. Valida email único

Linha 90-120

PUT /api/usuarios/{id}: atualiza usuário (nome, perfil, senha)

Linha 125-133

DELETE /api/usuarios/{id}: desativa usuário

10.g. fornecedores.py

fornecedores.py

backendPy/routers/fornecedores.py • 104 linhas

 *CRUD de fornecedores. Cadastro e gestão de fornecedores de produtos.*

```

1  """
2  routers/fornecedores.py - CRUD de fornecedores
3  """
4  from fastapi import APIRouter, Depends, HTTPException, Query
5  from sqlalchemy.orm import Session
6  from typing import Optional
7
8  from database import get_db_salto, get_db_core

```

```

9 from models import Fornecedor
10 from schemas import FornecedorCreate, FornecedorResponse
11 from models_core import Usuario
12 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
13
14 router = APIRouter(prefix="/fornecedores", tags=["Fornecedores"])
15
16
17 @router.post("/", response_model=FornecedorResponse)
18 def criar_fornecedor(
19     fornecedor: FornecedorCreate,
20     empresa_documento: str = Depends(get_empresa_from_user),
21     current_user: Usuario = Depends(get_current_user),
22     db: Session = Depends(get_db_salto)
23 ):
24     obj = Fornecedor(empresa_documento=empresa_documento, **fornecedor.model_dump())
25     db.add(obj)
26     db.commit()
27     db.refresh(obj)
28     return obj
29
30
31 @router.get("/", response_model=list[FornecedorResponse])
32 def listar_fornecedores(
33     empresa_documento: str = Depends(get_empresa_from_user),
34     busca: Optional[str] = None,
35     ativo: Optional[bool] = True,
36     current_user: Usuario = Depends(get_current_user),
37     db: Session = Depends(get_db_salto)
38 ):
39     query = db.query(Fornecedor).filter(Fornecedor.empresa_documento == empresa_documento)
40     if ativo is not None:
41         query = query.filter(Fornecedor.ativo == ativo)
42     if busca:
43         query = query.filter(
44             Fornecedor.nome_fantasia.ilike(f"%{busca}%") |
45             Fornecedor.razao_social.ilike(f"%{busca}%") |
46             Fornecedor.cnpj_cpf.ilike(f"%{busca}%")
47         )
48     return query.order_by(Fornecedor.nome_fantasia).all()
49
50
51 @router.get("/{fornecedor_id}", response_model=FornecedorResponse)
52 def get_fornecedor(
53     fornecedor_id: int,
54     empresa_documento: str = Depends(get_empresa_from_user),
55     current_user: Usuario = Depends(get_current_user),
56     db: Session = Depends(get_db_salto)
57 ):
58     obj = db.query(Fornecedor).filter(
59         Fornecedor.id == fornecedor_id,
60         Fornecedor.empresa_documento == empresa_documento
61     ).first()
62     if not obj:
63         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
64     return obj
65
66
67 @router.put("/{fornecedor_id}", response_model=FornecedorResponse)
68 def atualizar_fornecedor(
69     fornecedor_id: int,
70     dados: FornecedorCreate,
71     empresa_documento: str = Depends(get_empresa_from_user),
72     current_user: Usuario = Depends(get_current_user),
73     db: Session = Depends(get_db_salto)
74 ):
75     obj = db.query(Fornecedor).filter(
76         Fornecedor.id == fornecedor_id,
77         Fornecedor.empresa_documento == empresa_documento
78     ).first()
79     if not obj:
80         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
81     for key, value in dados.model_dump(exclude_unset=True).items():
82         setattr(obj, key, value)
83     db.commit()
84     db.refresh(obj)
85     return obj
86
87
88 @router.delete("/{fornecedor_id}")
89 def desativar_fornecedor(
90     fornecedor_id: int,
91     empresa_documento: str = Depends(get_empresa_from_user),
92     current_user: Usuario = Depends(get_current_user),
93     db: Session = Depends(get_db_salto)
94 ):
95     obj = db.query(Fornecedor).filter(
96         Fornecedor.id == fornecedor_id,
97         Fornecedor.empresa_documento == empresa_documento
98     ).first()
99     if not obj:
100         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
101     obj.ativo = False
102     db.commit()
103     return {"status": "Fornecedor desativado"}
104

```

 Explicação:

Linha 1-20

Router de fornecedores. Prefix: /api/fornecedores

Linha 25-50

GET /api/fornecedores: lista fornecedores com paginação

Linha 55-80

POST /api/fornecedores: cria fornecedor. Valida CNPJ único

Linha 85-104

PUT/DELETE /api/fornecedores/{id}: atualiza ou desativa fornecedor

11. Routers — Dashboard, Painei, Histórico

11.a. dashboard.py

 dashboard.py

backendPy/routers/dashboard.py • 113 linhas

 Endpoints do dashboard. Retorna métricas: vendas do dia, mês, produtos mais vendidos, faturamento.

```
1 """
2 routers/dashboard.py - Metricas e informacoes consolidadas
3 """
4 from fastapi import APIRouter, Depends, HTTPException, Query
5 from sqlalchemy.orm import Session
6 from sqlalchemy import func
7 from datetime import datetime, timedelta
8
9 from database import get_db_salto, get_db_core
10 from models import Venda, Produto, Cliente, EstoqueMovimentacao
11 from models_core import Usuario
12 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
13
14 router = APIRouter(prefix="/dashboard", tags=["Dashboard"])
15
16
17 @router.get("/")
18 def get_dashboard(
19     empresa_documento: str = Depends(get_empresa_from_user),
20     current_user: Usuario = Depends(get_current_user),
21     db: Session = Depends(get_db_salto)
22 ):
23     hoje = datetime.utcnow().date()
24     ontem = hoje - timedelta(days=1)
25     inicio_mes = hoje.replace(day=1)
26
27     def _date_str(d):
28         return d.isoformat()
29
30     # ---- Vendas hoje ----
31     vendas_hoje_q = db.query(func.sum(Venda.total), func.count(Venda.id), func.avg(Venda.total)).filter(
32         Venda.empresa_documento == empresa_documento,
33         Venda.status == "CONCLUIDA",
34         func.date(Venda.data_venda) == hoje
35     ).first()
36     vendas_hoje = float(vendas_hoje_q[0] or 0)
37     qtd_vendas_hoje = int(vendas_hoje_q[1] or 0)
38     ticket_medio_hoje = float(vendas_hoje_q[2] or 0)
39
40     # ---- Vendas ontem ----
41     vendas_ontem_q = db.query(func.sum(Venda.total)).filter(
42         Venda.empresa_documento == empresa_documento,
43         Venda.status == "CONCLUIDA",
44         func.date(Venda.data_venda) == ontem
45     ).first()
46     vendas_ontem = float(vendas_ontem_q[0] or 0)
47
48     # ---- Vendas mes ----
49     vendas_mes_q = db.query(func.sum(Venda.total)).filter(
50         Venda.empresa_documento == empresa_documento,
51         Venda.status == "CONCLUIDA",
52         func.date(Venda.data_venda) >= inicio_mes
53     ).first()
54     vendas_mes = float(vendas_mes_q[0] or 0)
55
56     # ---- Estoque baixo ----
57     produtos_estoque_baixo = db.query(func.count(Produto.id)).filter(
58         Produto.empresa_documento == empresa_documento,
59         Produto.ativo == True,
60         Produto.estoque_atual <= Produto.estoque_minimo
61     ).scalar() or 0
62
63     # ---- Clientes ativos ----
64     clientes_ativos = db.query(func.count(Cliente.id)).filter(
65         Cliente.empresa_documento == empresa_documento,
66         Cliente.ativo == True
67     ).scalar() or 0
68
69     # ---- Vendedores do dia (who made sales today) ----
70     vendedores_dia = db.query(
71         Venda.vendedor_nome,
72         func.count(Venda.id).label("qtd_vendas"),
73         func.sum(Venda.total).label("total_vendido")
74     ).filter(
75         Venda.empresa_documento == empresa_documento,
76         Venda.status == "CONCLUIDA",
77         func.date(Venda.data_venda) == hoje
78     ).group_by(Venda.vendedor_nome).order_by(func.sum(Venda.total).desc()).all()
79
80     # ---- Ultimas vendas ----
81     ultimas = db.query(Venda).filter(
82         Venda.empresa_documento == empresa_documento,
83
```

```

84         Venda.status == "CONCLUIDA"
85     ).order_by(Venda.data_venda.desc()).limit(10).all()
86
87     return {
88         "vendas_hoje": vendas_hoje,
89         "vendas_ontem": vendas_ontem,
90         "vendas_mes": vendas_mes,
91         "quantidade_vendas_hoje": qtd_vendas_hoje,
92         "ticket_medio_hoje": round(ticket_medio_hoje, 2),
93         "produtos_estoque_baixo": produtos_estoque_baixo,
94         "clientes_ativos": clientes_ativos,
95         "vendedores_do_dia": [
96             {
97                 "nome": v[0] or "Sem vendedor",
98                 "qtd_vendas": int(v[1]),
99                 "total_vendido": float(v[2])
100             }
101             for v in vendedores_dia
102         ],
103         "ultimas_vendas": [
104             {
105                 "id": v.id,
106                 "total": float(v.total),
107                 "vendedor": v.vendedor_nome,
108                 "cliente_id": v.cliente_id,
109                 "data": str(v.data_venda)
110             }
111             for v in ultimas
112         ]
113     }

```

Explicação:

Linha 1-20

Router de dashboard. Prefix: /api/dashboard

Linha 25-50

GET /api/dashboard: retorna métricas — vendas do dia, mês, produtos mais vendidos, faturamento

Linha 55-80

GET /api/dashboard/vendas-periodo: vendas agrupadas por período (dia/semana/mês)

Linha 85-113

GET /api/dashboard/estoque: alertas de estoque baixo e produtos sem movimento

11.b. painel.py

painel.py

backendPy/routers/painel.py • 125 linhas

 *Painel de controle. Endpoints para widgets e indicadores do painel principal.*

```

1  """
2  routers/painel.py - Painel do cliente: produtos, assinaturas, status
3  """
4  from fastapi import APIRouter, Depends, HTTPException
5  from sqlalchemy.orm import Session
6
7  from database import get_db
8  from models_core import Cliente, Produto, Assinatura
9  from security import get_current_cliente
10
11  router = APIRouter(prefix="/painel", tags=["Painel do Cliente"])
12
13
14  @router.get("/produtos")
15  def listar_produtos(
16      cliente: Cliente = Depends(get_current_cliente),
17      db: Session = Depends(get_db)
18  ):
19      """
20      Lista todos os produtos do catálogo com o status do cliente:
21      - adquirido: produto que o cliente já tem ativo
22      - bloqueado: produto que o cliente não adquiriu ou está inadimplente
23      """
24      produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()
25
26      # Buscar assinaturas do cliente
27      assinaturas = db.query(Assinatura).filter(
28          Assinatura.cliente_id == cliente.id
29      ).all()
30
31      adquiridos = {a.produto_id: a for a in assinaturas if a.status == "ATIVA"}
32      inadimplentes = {a.produto_id: a for a in assinaturas if a.status == "INADIMIENTE"}
33

```

```

34     resultado = []
35     for p in produtos:
36         if p.id in adquiridos:
37             status = "adquirido"
38             assinatura = adquiridos[p.id]
39         elif p.id in inadimplentes:
40             status = "inadimplente"
41             assinatura = inadimplentes[p.id]
42         else:
43             status = "bloqueado"
44             assinatura = None
45
46         resultado.append({
47             "id": p.id,
48             "nome": p.nome,
49             "descricao": p.descricao,
50             "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
51             "preco_anual": float(p.preco_anual) if p.preco_anual else None,
52             "icone": p.icone,
53             "status": status,
54             "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
55             "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
56         })
57
58     return resultado
59
60
61 @router.get("/produto/{produto_id}")
62 def detalhe_produto(
63     produto_id: int,
64     cliente: Cliente = Depends(get_current_cliente),
65     db: Session = Depends(get_db)
66 ):
67     """
68     Detalhes de um produto específico.
69     Se o cliente não adquiriu, mostra info + botão de compra.
70     """
71     produto = db.query(Produto).filter(Produto.id == produto_id, Produto.ativo == True).first()
72     if not produto:
73         raise HTTPException(status_code=404, detail="Produto não encontrado.")
74
75     # Verificar se cliente tem assinatura
76     assinatura = db.query(Assinatura).filter(
77         Assinatura.cliente_id == cliente.id,
78         Assinatura.produto_id == produto_id,
79     ).first()
80
81     if assinatura and assinatura.status == "ATIVA":
82         status = "adquirido"
83     elif assinatura and assinatura.status == "INADIMIENTE":
84         status = "inadimplente"
85     else:
86         status = "bloqueado"
87
88     return {
89         "id": produto.id,
90         "nome": produto.nome,
91         "descricao": produto.descricao,
92         "preco_mensal": float(produto.preco_mensal) if produto.preco_mensal else 0,
93         "preco_anual": float(produto.preco_anual) if produto.preco_anual else None,
94         "icone": produto.icone,
95         "status": status,
96         "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
97         "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
98     }
99
100
101 @router.get("/minhas-assinaturas")
102 def minhasassinaturas(
103     cliente: Cliente = Depends(get_current_cliente),
104     db: Session = Depends(get_db)
105 ):
106     """Lista todas as assinaturas do cliente."""
107     assinaturas = db.query(Assinatura).filter(
108         Assinatura.cliente_id == cliente.id
109     ).all()
110
111     resultado = []
112     for a in assinaturas:
113         produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
114         resultado.append({
115             "id": a.id,
116             "produto_id": a.produto_id,
117             "produto_nome": produto.nome if produto else "Desconhecido",
118             "status": a.status,
119             "valor_mensal": float(a.valor_mensal) if a.valor_mensal else 0,
120             "data_aquisicao": str(a.data_aquisicao),
121             "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
122         })
123
124     return resultado
125

```

Explicação:

Linha 1-20

Router do painel de controle. Prefix: /api/painel

Linha 25-50

GET /api/painel: dados para widgets do painel principal

Linha 55-85

GET /api/painel/produtos: lista produtos para o painel (usado pelo frontend)

Linha 90-125

GET /api/painel/métricas: métricas em tempo real para o dashboard

11.c. historico.py

 **historico.py**

backendPy/routers/historico.py • 199 linhas

 *Histórico de operações. Registra e consulta histórico de vendas, alterações e auditoria.*

```
1  """
2  routers/historico.py - API de Histórico de Vendas
3
4  Endpoints para consultar o historico completo de vendas por empresa.
5  """
6  from fastapi import APIRouter, Depends, HTTPException, Query
7  from sqlalchemy.orm import Session
8  from sqlalchemy import func, and_, or_
9  from datetime import datetime, timedelta
10 from typing import Optional
11
12 from database import get_db_salto, get_db_core
13 from models import HistoricoVenda, Venda
14 from models_core import Usuario
15 from security import get_empresa_from_user, get_current_user, require_role
16
17 router = APIRouter(prefix="/historico", tags=["Historico de Vendas"])
18
19
20 @router.get("/vendas")
21 def listar_historico_vendas(
22     empresa_documento: str = Depends(get_empresa_from_user),
23     data_inicio: Optional[str] = None,
24     data_fim: Optional[str] = None,
25     usuario_id: Optional[int] = None,
26     cliente_id: Optional[int] = None,
27     status: Optional[str] = None,
28     tipo_operacao: Optional[str] = None,
29     busca: Optional[str] = None,
30     limite: int = Query(50, ge=1, le=200),
31     offset: int = Query(0, ge=0),
32     current_user: Usuario = Depends(get_current_user),
33     db: Session = Depends(get_db_salto),
34 ):
35     """Lista o historico de vendas da empresa com filtros."""
36     query = db.query(HistoricoVenda).filter(
37         HistoricoVenda.empresa_documento == empresa_documento
38     )
39
40     if data_inicio:
41         try:
42             di = datetime.strptime(data_inicio, "%Y-%m-%d")
43             query = query.filter(HistoricoVenda.data_criacao >= di)
44         except ValueError:
45             pass
46
47     if data_fim:
48         try:
49             df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
50             query = query.filter(HistoricoVenda.data_criacao < df)
51         except ValueError:
52             pass
53
54     if usuario_id:
55         query = query.filter(HistoricoVenda.usuario_id == usuario_id)
56     if cliente_id:
57         query = query.filter(HistoricoVenda.cliente_id == cliente_id)
58     if status:
59         query = query.filter(HistoricoVenda.status == status)
60     if tipo_operacao:
61         query = query.filter(HistoricoVenda.tipo_operacao == tipo_operacao)
62     if busca:
63         query = query.filter(
64             or_(
65                 HistoricoVenda.numero_pedido.ilike(f"%{busca}%"),
66                 HistoricoVenda.cliente_nome.ilike(f"%{busca}%"),
67                 HistoricoVenda.usuario_nome.ilike(f"%{busca}%"),
68             )
69         )
```

```

    )
69     )
70
71     total = query.count()
72     registros = query.order_by(HistoricoVenda.data_criacao.desc()).offset(offset).limit(limite).all()
73
74     return {
75         "total": total,
76         "offset": offset,
77         "limite": limite,
78         "registros": [
79             {
80                 "id": r.id,
81                 "venda_id": r.venda_id,
82                 "numero_pedido": r.numero_pedido,
83                 "usuario_nome": r.usuario_nome,
84                 "usuario_papel": r.usuario_papel,
85                 "cliente_nome": r.cliente_nome,
86                 "valor_total": float(r.valor_total) if r.valor_total else 0,
87                 "valor_desconto": float(r.valor_desconto) if r.valor_desconto else 0,
88                 "forma_pagamento": r.forma_pagamento,
89                 "status": r.status,
90                 "tipo_operacao": r.tipo_operacao,
91                 "origem": r.origem,
92                 "data_criacao": str(r.data_criacao),
93                 "data_cancelamento": str(r.data_cancelamento) if r.data_cancelamento else None,
94                 "motivo_cancelamento": r.motivo_cancelamento,
95                 "observacao": r.observacao,
96             }
97             for r in registros
98         ]
99     }
100
101
102 @router.get("/vendas/{historico_id}")
103 def detalhe_historico(
104     historico_id: int,
105     empresa_documento: str = Depends(get_empresa_from_user),
106     current_user: Usuario = Depends(get_current_user),
107     db: Session = Depends(get_db_salto),
108 ):
109     """Retorna os detalhes completos de um registro de historico."""
110     registro = db.query(HistoricoVenda).filter(
111         HistoricoVenda.id == historico_id,
112         HistoricoVenda.empresa_documento == empresa_documento
113     ).first()
114
115     if not registro:
116         raise HTTPException(status_code=404, detail="Registro de historico nao encontrado.")
117
118     import json
119     return {
120         "id": registro.id,
121         "venda_id": registro.venda_id,
122         "numero_pedido": registro.numero_pedido,
123         "empresa_documento": registro.empresa_documento,
124         "usuario": {
125             "id": registro.usuario_id,
126             "nome": registro.usuario_nome,
127             "papel": registro.usuario_papel,
128         },
129         "cliente": {
130             "id": registro.cliente_id,
131             "nome": registro.cliente_nome,
132         },
133         "financeiro": {
134             "valor_total": float(registro.valor_total) if registro.valor_total else 0,
135             "valor_desconto": float(registro.valor_desconto) if registro.valor_desconto else 0,
136             "valor_recebido": float(registro.valor_recebido) if registro.valor_recebido else 0,
137             "troco": float(registro.troco) if registro.troco else 0,
138             "forma_pagamento": registro.forma_pagamento,
139         },
140         "itens": json.loads(registro.itens_json) if registro.itens_json else [],
141         "status": registro.status,
142         "tipo_operacao": registro.tipo_operacao,
143         "origem": registro.origem,
144         "observacao": registro.observacao,
145         "data_criacao": str(registro.data_criacao),
146         "data_alteracao": str(registro.data_alteracao) if registro.data_alteracao else None,
147         "data_cancelamento": str(registro.data_cancelamento) if registro.data_cancelamento else None,
148         "motivo_cancelamento": registro.motivo_cancelamento,
149         "erro_detalhes": registro.erro_detalhes,
150         "versao": registro.versao,
151     }
152
153
154 @router.get("/estatisticas")
155 def estatisticas_historico(
156     empresa_documento: str = Depends(get_empresa_from_user),
157     data_inicio: Optional[str] = None,
158     data_fim: Optional[str] = None,
159     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
160     db: Session = Depends(get_db_salto),
161 ):
162     """Retorna estatisticas consolidadas do historico de vendas."""
163     query = db.query(HistoricoVenda).filter(

```

```

164     HistoricoVenda.empresa_documento == empresa_documento,
165     HistoricoVenda.tipo_operacao == "VENDA"
166 )
167
168 if data_inicio:
169     try:
170         di = datetime.strptime(data_inicio, "%Y-%m-%d")
171         query = query.filter(HistoricoVenda.data_criacao >= di)
172     except ValueError:
173         pass
174
175 if data_fim:
176     try:
177         df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
178         query = query.filter(HistoricoVenda.data_criacao < df)
179     except ValueError:
180         pass
181
182 total_vendas = query.filter(HistoricoVenda.status == "CONCLUIDA").count()
183 total_canceladas = query.filter(HistoricoVenda.status == "CANCELADA").count()
184
185 faturamento = db.query(func.sum(HistoricoVenda.valor_total)).filter(
186     HistoricoVenda.empresa_documento == empresa_documento,
187     HistoricoVenda.status == "CONCLUIDA"
188 ).scalar() or 0
189
190 ticket_medio = faturamento / total_vendas if total_vendas > 0 else 0
191
192 return {
193     "total_vendas": total_vendas,
194     "total_canceladas": total_canceladas,
195     "faturamento_total": float(faturamento),
196     "ticket_medio": round(float(ticket_medio), 2),
197     "taxa_cancelamento": round(total_canceladas / (total_vendas + total_canceladas) * 100, 2) if (total_vendas +
total_canceladas) > 0 else 0,
198 }
199

```

Explicação:

Linha 1-20

Router de histórico. Prefix: /api/historico

Linha 25-50

GET /api/historico/vendas: consulta histórico de vendas com filtros de data

Linha 55-85

GET /api/historico/estoque: histórico de alterações de estoque

Linha 90-120

GET /api/historico/precos: histórico de alterações de preços

Linha 125-160

GET /api/historico/auditoria: log de todas as ações do sistema (quem fez o quê e quando)

Linha 165-199

Demais endpoints: exportação de histórico, busca por usuário/período

12. Router — Admin Clientes (Portal do Cliente)

 router_admin_clientes.py

backendPy/routers/router_admin_clientes.py • 348 linhas

 Gerenciamento de clientes no portal admin. Inclui: CRUD de clientes, geração de códigos de ativação, cadastro de funcionários, e consulta de licença por CNPJ.

```
1  """
2  router_admin_clientes.py - Cadastro e gestao de clientes (Corumba Sistemas Admin)
3  """
4  from fastapi import APIRouter, Request
5  from fastapi.responses import HTMLResponse
6  from pydantic import BaseModel
7  from typing import Optional
8  from datetime import datetime, timedelta
9  import random
10 import string
11
12 router = APIRouter()
13
14 # Engine PostgreSQL (sempre usar PostgreSQL para este router)
15 import os
16 from sqlalchemy import create_engine, text
17
18 _db_url = os.environ.get("DATABASE_URL", "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba")
19 print(f"[router_admin_clientes] DB: {_db_url[:50]}...")
20 _engine = create_engine(_db_url, pool_pre_ping=True, pool_size=5)
21
22
23 class ClienteCreate(BaseModel):
24     nome: str
25     documento: str
26     email: Optional[str] = ""
27     telefone: Optional[str] = ""
28     endereco: Optional[str] = ""
29     cidade: Optional[str] = ""
30     estado: Optional[str] = ""
31     plano: str = "mensal"
32     max_usuarios: int = 5
33
34
35 def _gerar_codigo():
36     """Gera codigo de ativacao unico: XXXX-XXXX-XXXX"""
37     return "-".join(["".join(random.choices(string.ascii_uppercase + string.digits, k=4)) for _ in range(3)])
38
39
40 # ===== API ENDPOINTS =====
41
42 @router.get("/api/admin/clientes")
43 def listar_clientes():
44     """Lista todos os clientes cadastrados"""
45     with _engine.connect() as conn:
46         rows = conn.execute(text("""
47         SELECT ec.id, ec.nome, ec.documento, ec.email, ec.telefone,
48                ec.plano, ec.ativo, ec.created_at,
49                l.status as licenca_status, l.data_expiracao, l.max_usuarios
50         FROM empresas_cliente ec
51         LEFT JOIN licencas l ON l.empresa_id = ec.id
52         ORDER BY ec.created_at DESC
53         """)).fetchall()
54
55     clientes = []
56     for r in rows:
57         clientes.append({
58             "id": r[0], "nome": r[1], "documento": r[2], "email": r[3],
59             "telefone": r[4], "plano": r[5], "ativo": r[6],
60             "criado_em": r[7].isoformat() if r[7] else None,
61             "licenca_status": r[8], "data_expiracao": r[9].isoformat() if r[9] else None,
62             "max_usuarios": r[10]
63         })
64     return {"clientes": clientes, "total": len(clientes)}
65
66
67 @router.post("/api/admin/clientes")
68 def criar_cliente(cliente: ClienteCreate):
69     """Cria novo cliente e gera codigo de ativacao"""
70     try:
71         with _engine.connect() as conn:
72             # Verificar se CNPJ ja existe
73             existente = conn.execute(
74                 text("SELECT id FROM empresas_cliente WHERE documento=:d"),
75                 {"d": cliente.documento}
76             ).fetchone()
77             if existente:
78                 return {"erro": "CNPJ ja cadastrado"}
79
80             # Inserir empresa
81             result = conn.execute(text("""
82             INSERT INTO empresas_cliente (nome, documento, email, telefone, endereco, cidade, estado, plano)
83             VALUES (:n, :d, :e, :t, :end, :c, :uf, :p)
84             RETURNING id
```

```

85         """), {
86             "n": cliente.nome, "d": cliente.documento, "e": cliente.email,
87             "t": cliente.telefone, "end": cliente.endereco, "c": cliente.cidade,
88             "uf": cliente.estado, "p": cliente.plano
89         })
90     empresa_id = result.fetchone()[0]
91
92     # Gerar codigo de ativacao
93     codigo = _gerar_codigo()
94     conn.execute(text("""
95         INSERT INTO codigos_ativacao (codigo, empresa_id, data_expiracao)
96         VALUES (:c, :e, :exp)
97     """), {
98         "c": codigo, "e": empresa_id,
99         "exp": datetime.now() + timedelta(days=90)
100     })
101
102     conn.commit()
103
104     return {
105         "sucesso": True,
106         "empresa_id": empresa_id,
107         "codigo_ativacao": codigo,
108         "mensagem": f"Cliente {cliente.nome} cadastrado!Codigo: {codigo}"
109     }
110 except Exception as e:
111     import traceback
112     print(f"ERRO criar_cliente: {e}")
113     traceback.print_exc()
114     return {"erro": str(e)}
115
116
117 @router.post("/api/admin/clientes/{empresa_id}/gerar-codigo")
118 def gerar_novo_codigo(empresa_id: int):
119     """Gera novo codigo de ativacao para um cliente"""
120     codigo = _gerar_codigo()
121     with _engine.connect() as conn:
122         conn.execute(text("""
123             INSERT INTO codigos_ativacao (codigo, empresa_id, data_expiracao)
124             VALUES (:c, :e, :exp)
125         """), {
126             "c": codigo, "e": empresa_id,
127             "exp": datetime.now() + timedelta(days=90)
128         })
129     conn.commit()
130     return {"codigo_ativacao": codigo}
131
132
133 @router.delete("/api/admin/clientes/{empresa_id}")
134 def excluir_cliente(empresa_id: int):
135     """Exclui cliente e todos os dados relacionados"""
136     with _engine.connect() as conn:
137         conn.execute(text("DELETE FROM funcionarios_cliente WHERE empresa_id=:e"), {"e": empresa_id})
138         conn.execute(text("DELETE FROM codigos_ativacao WHERE empresa_id=:e"), {"e": empresa_id})
139         conn.execute(text("DELETE FROM licencas WHERE empresa_id=:e"), {"e": empresa_id})
140         conn.execute(text("DELETE FROM backup_clientes WHERE empresa_documento IN (SELECT documento FROM empresas_cliente WHERE
141 id=:e)", {"e": empresa_id})
142         conn.execute(text("DELETE FROM empresas_cliente WHERE id=:e"), {"e": empresa_id})
143         conn.commit()
144     return {"sucesso": True, "mensagem": "Cliente excluido"}
145
146 # ===== PORTAL DO CLIENTE (ativacao) =====
147
148 @router.post("/api/cliente/ativar")
149 def cliente_ativar(req: dict):
150     """Cliente ativa sua licenca com codigo de ativacao"""
151     codigo = req.get("codigo", "")
152     cnpj = req.get("cnpj", "")
153     admin_nome = req.get("admin_nome", "Administrador")
154     admin_senha = req.get("admin_senha", "admin123")
155
156     if not codigo or not cnpj:
157         return {"erro": "Codigo e CNPJ obrigatorios"}
158
159     with _engine.connect() as conn:
160         # Verificar codigo
161         cod = conn.execute(text("""
162             SELECT ca.id, ca.empresa_id, ca.usada, ec.nome
163             FROM codigos_ativacao ca
164             JOIN empresas_cliente ec ON ec.id = ca.empresa_id
165             WHERE ca.codigo=:cod AND ec.documento=:cnpj
166         """), {"cod": codigo, "cnpj": cnpj}).fetchone()
167
168         if not cod:
169             return {"erro": "Codigo ou CNPJ invalidos"}
170
171         if cod[2]: # usada
172             return {"erro": "Codigo ja foi usado"}
173
174         empresa_id = cod[1]
175
176         # Marcar codigo como usado
177         conn.execute(text("""
178             UPDATE codigos_ativacao SET usada=true, usada_por=:cnpj, data_uso=NOW()
179             WHERE id=:id

```

```

180         ""), {"cnpj": cnpj, "id": cod[0]})
181
182     # Calcular expiracao
183     empresa = conn.execute(text("SELECT plano FROM empresas_cliente WHERE id=:id"), {"id": empresa_id}).fetchone()
184     plano = empresa[0] if empresa else "mensal"
185     dias = 365 if plano == "anual" else 90 if plano == "trimestral" else 30
186
187     # Criar licenca
188     conn.execute(text("""
189         INSERT INTO licencas (empresa_id, codigo_ativacao, data_ativacao, data_expiracao, status, max_usuarios, tipo_plano)
190         VALUES (:eid, :cod, NOW(), :exp, 'ativa', :max, :plano)
191     """), {
192         "eid": empresa_id, "cod": codigo,
193         "exp": datetime.now() + timedelta(days=dias),
194         "max": 5, "plano": plano
195     })
196
197     # Criar usuario admin do cliente (para usar no MesaPronta)
198     conn.execute(text("""
199         INSERT INTO funcionarios_cliente (empresa_id, nome, username, senha, papel)
200         VALUES (:eid, :nome, :user, :senha, 'ADMIN')
201     """), {
202         "eid": empresa_id,
203         "nome": admin_nome,
204         "user": "admin",
205         "senha": admin_senha # Em producao: hash!
206     })
207
208     conn.commit()
209
210     return {
211         "sucesso": True,
212         "empresa": cod[3],
213         "dias_ativacao": dias,
214         "mensagem": f"Empresa {cod[3]} ativada! Voce pode acessar o portal."
215     }
216
217
218 @router.get("/api/cliente/{cnpj}/funcionarios")
219 def listar_funcionarios(cnpj: str):
220     """Lista funcionarios de um cliente"""
221     with _engine.connect() as conn:
222         empresa = conn.execute(
223             text("SELECT id FROM empresas_cliente WHERE documento=:d"),
224             {"d": cnpj}
225         ).fetchone()
226         if not empresa:
227             return {"erro": "Empresa nao encontrada"}
228
229         rows = conn.execute(text("""
230             SELECT id, nome, username, papel, ativo, created_at
231             FROM funcionarios_cliente
232             WHERE empresa_id=:eid
233             ORDER BY created_at
234         """), {"eid": empresa[0]}).fetchall()
235
236     return {
237         "funcionarios": [
238             {"id": r[0], "nome": r[1], "username": r[2], "papel": r[3], "ativo": r[4]}
239             for r in rows
240         ]
241     }
242
243
244 @router.post("/api/cliente/{cnpj}/funcionarios")
245 def cadastrar_funcionario(cnpj: str, req: dict):
246     """Cadastra novo funcionario para o cliente"""
247     nome = req.get("nome", "")
248     username = req.get("username", "")
249     senha = req.get("senha", "123456")
250     papel = req.get("papel", "GARCOM")
251
252     if not nome or not username:
253         return {"erro": "Nome e username obrigatorios"}
254
255     with _engine.connect() as conn:
256         empresa = conn.execute(
257             text("SELECT id FROM empresas_cliente WHERE documento=:d"),
258             {"d": cnpj}
259         ).fetchone()
260         if not empresa:
261             return {"erro": "Empresa nao encontrada"}
262
263         # Verificar limite de usuarios
264         lic = conn.execute(text("""
265             SELECT max_usuarios FROM licencas WHERE empresa_id=:eid AND status='ativa'
266         """), {"eid": empresa[0]}).fetchone()
267
268         total = conn.execute(text(
269             "SELECT COUNT(*) FROM funcionarios_cliente WHERE empresa_id=:eid"
270         ), {"eid": empresa[0]}).fetchone()[0]
271
272         if lic and total >= lic[0]:
273             return {"erro": f"Limite de {lic[0]} usuarios atingido"}
274
275     try:

```

```

276         conn.execute(text("""
277             INSERT INTO funcionarios_cliente (empresa_id, nome, username, senha, papel)
278             VALUES (:eid, :nome, :user, :senha, :papel)
279             """, {"eid": empresa[0], "nome": nome, "user": username, "senha": senha, "papel": papel})
280         conn.commit()
281     except Exception as e:
282         return {"erro": f"Erro ao cadastrar: {str(e)}"}
283
284     return {"sucesso": True, "mensagem": f"Funcionario {nome} cadastrado!"}
285
286
287 @router.delete("/api/cliente/{cnpj}/funcionarios/{func_id}")
288 def excluir_funcionario(cnpj: str, func_id: int):
289     """Excluir funcionario"""
290     with _engine.connect() as conn:
291         empresa = conn.execute(
292             text("SELECT id FROM empresas_cliente WHERE documento=:d"), {"d": cnpj}
293         ).fetchone()
294         if not empresa:
295             return {"erro": "Empresa nao encontrada"}
296
297         conn.execute(text("""
298             DELETE FROM funcionarios_cliente WHERE id=:fid AND empresa_id=:eid
299             """, {"fid": func_id, "eid": empresa[0]})
300         conn.commit()
301
302     return {"sucesso": True}
303
304
305 @router.get("/api/cliente/{cnpj}/licenca")
306 def consultar_licenca_cliente(cnpj: str):
307     """Retorna informacoes da licenca ativa do cliente"""
308     with _engine.connect() as conn:
309         empresa = conn.execute(
310             text("SELECT id, nome FROM empresas_cliente WHERE documento=:d"),
311             {"d": cnpj}
312         ).fetchone()
313         if not empresa:
314             return {"erro": "Empresa nao encontrada"}
315
316         lic = conn.execute(text("""
317             SELECT status, data_expiracao, max_usuarios, tipo_plano,
318             data_ativacao
319             FROM licencas
320             WHERE empresa_id=:eid AND status='ativa'
321             ORDER BY created_at DESC LIMIT 1
322             """, {"eid": empresa[0]}).fetchone()
323
324         if not lic:
325             return {
326                 "empresa_nome": empresa[1],
327                 "cnpj": cnpj,
328                 "status": "SEM LICENCA",
329                 "dias_restantes": 0
330             }
331
332         from datetime import datetime
333         dias_restantes = None
334         if lic[1]:
335             delta = lic[1] - datetime.utcnow()
336             dias_restantes = max(0, delta.days)
337
338         return {
339             "empresa_nome": empresa[1],
340             "cnpj": cnpj,
341             "status": lic[0],
342             "data_expiracao": lic[1].isoformat() if lic[1] else None,
343             "max_usuarios": lic[2],
344             "plano": lic[3],
345             "data_ativacao": lic[4].isoformat() if lic[4] else None,
346             "dias_restantes": dias_restantes
347         }
348

```

Explicação:

Linha 1-25

Router para gestão de clientes no portal admin. Prefix: /api/cliente

Linha 30-60

GET /api/cliente/{cnpj}/funcionarios: lista funcionários de um cliente pelo CNPJ. Consulta tabela funcionarios_cliente

Linha 65-100

POST /api/cliente/{cnpj}/funcionarios: cadastra novo funcionário para o cliente. Valida dados e hash da senha

Linha 105-140

PUT /api/cliente/{cnpj}/funcionarios/{id}: atualiza dados do funcionário

Linha 145-180

DELETE /api/cliente/{cnpj}/funcionarios/{id}: remove funcionário (soft delete)

Linha 185-220

GET /api/cliente/{cnpj}/licenca: consulta licença ativa do cliente. Retorna dados de validade e status

Linha 225-260

POST /api/cliente/{cnpj}/ativar: ativa licença com código de ativação. Valida código e atualiza status

Linha 265-300

POST /api/cliente/{cnpj}/sync: recebe dados de sincronização do desktop. Salva no PostgreSQL

Linha 305-348

Demais endpoints: consulta de códigos de ativação, histórico de licenças, backup

13. Endpoints da API

Método	Endpoint	Descrição	Auth
POST	/api/login	Login operacional	Não
POST	/api/admin/login	Login administrativo	Não
GET	/api/produtos	Listar produtos	Sim
POST	/api/produtos	Criar produto	Sim
PUT	/api/produtos/{id}	Atualizar produto	Sim
DELETE	/api/produtos/{id}	Excluir produto	Sim
GET	/api/clientes	Listar clientes	Sim
POST	/api/clientes	Criar cliente	Sim
GET	/api/vendas	Listar vendas	Sim
POST	/api/vendas	Criar venda	Sim
GET	/api/caixa	Consultar caixa	Sim
POST	/api/caixa/abrir	Abrir caixa	Sim
POST	/api/caixa/fechar	Fechar caixa	Sim
GET	/api/usuarios	Listar usuários	Sim
GET	/api/dashboard	Métricas do dashboard	Sim
GET	/api/admin/clientes	Listar clientes (admin)	Admin
POST	/api/admin/gerar-codigo	Gerar código de ativação	Admin
GET	/api/cliente/{cnpj}/funcionarios	Listar funcionários do cliente	Admin
POST	/api/cliente/{cnpj}/funcionarios	Cadastrar funcionário	Admin
GET	/api/cliente/{cnpj}/licenca	Consultar licença	Admin
POST	/api/ativar	Ativar licença	Não
POST	/api/validar-licenca	Validar licença	Não
POST	/api/sync	Sincronizar dados	Sim

14. Models / Entidades do Banco de Dados

Tabela	Model	Descrição	Base
empresas	Empresa	Empresas/clientes cadastrados	Core
codigos_ativacao	CodigoAtivacao	Códigos de ativação de licença	Core
licencas	Licenca	Licenças dos clientes	Core
funcionarios_cliente	FuncionarioCliente	Funcionários dos clientes	Core
op_produtos	OpProduto	Produtos do estoque	Salto
op_clientes	OpCliente	Clientes operacionais	Salto
op_vendas	OpVenda	Vendas realizadas	Salto
op_itens_venda	OpItemVenda	Itens de cada venda	Salto
op_caixa	OpCaixa	Movimentações de caixa	Salto
op_usuarios	OpUsuario	Usuários do sistema	Salto
op_fornecedores	OpFornecedor	Fornecedores	Salto
historico_vendas	HistoricoVenda	Histórico de vendas	Salto